OSCAR - Open Source Copter for Academic Research

Design Document

sddec26-05

Client - Matthew Smith

Advisor - Nathan Neihart

Team Members:

Matthew Smith - Team Leader

Collin Van Weelden - Hardware Lead

Paul Zell - Embedded Systems Lead

Jessica Reiff - Controls Lead

Grace White - Embedded Communications Lead

Chloe Atwood - Software and Version Control Lead

Grace Finger - Software and Embedded Integration Lead

Team Email: sddec26-05@iastate.edu

Team Website: https://sddec26-05.sd.ece.iastate.edu/

Revised: 05/03/26 - Version 1

Executive Summary

Drones are an accessible entry point into embedded systems and control theory, but learners are often locked out of how they actually work. The existing landscape of open source designs are fragmented, open firmware typically runs on closed PCB's, and open hardware rarely includes the firmware or software needed to fly it. No single integrated reference design exists.

There are two main design requirements shaping OSCAR. The first is fulfilling our duty of being open source with our work. Every schematic, PCB layout, parts list, and line of code is published openly, so anyone with the curiosity to learn can study, modify, and rebuild the entire system end to end. The second deliverable is the design itself, achieving manual steady level flight and accurate responsiveness through pilot commands utilizing the ExpressLRS protocol.

OSCAR is organized across three interconnected subsystems: the hardware, firmware, and software. The hardware consists of three separate PCBs, including the flight controller, which houses an STM32 microcontroller with an onboard IMU, barometer, and USB interface. A GNSS board with a passive patch antenna designed for 1.575 GHz. Finally, a dual-ESC board with 2oz copper pours to handle up to 50A motor pull. The firmware subsystem is designed around an 8 kHz PID loop fed by SPI communication with the ICM-40609 IMU, alongside I2C peripherals for the BNO055 magnetometer and ENS220 barometer, DSHOT-600 motor command output, ExpressLRS radio input, and a dual virtual COM port USB stack via TinyUSB for both command-response and continuous data streaming. The software subsystem includes the desktop application, OSCAR Ground Station, built on Electron with a native C module using libserialport, Node-API, and CRC-16/CCITT error validation, and designed to provide real-time 3D orientation visualization, PID tuning, motor testing, sensor calibration over USB, and similar drone configurations.

As of May 3rd, 2026, the project's current state includes the hardware team having successfully designed and assembled all three PCBs mentioned earlier. The flight controller has satisfied its requirements of voltage regulation, MCU startup, and communication over USB. The GNSS board has validated communication. However, validation testing of GNSS receivers across different environments has not yet been completed. The ESC design satisfied its requirements for voltage regulation, MCU startup, Gate drivers, and MOSFET handling. The firmware has successfully implemented the DSHOT protocol at 600 kHz, and USB command-response communication is functional between the firmware and the desktop application. The software team has implemented several UI pages, many of which display live flight controller data. On top of this, the software team has developed OSCAR Docs, the project's documentation website, which is currently live and is hosted by GitHub Pages. This site also holds the most recent version of the OSCAR Ground Station Desktop Application including the instructions on how to install it from the zip provided on the site.

We are currently basing our design on a well-established application that effectively meets user needs, using it as a benchmark for usability and functionality. In the future we are also planning on incorporating user feedback and testing to ensure our implementation aligns with the specific needs of our target users.

The next steps ahead for each subteam include the following. The hardware team plans to include a new revision for each PCB, with the flight controller condensed, including more I/O functionality, reducing redundant sensors. The GNSS board plans to include an added LNA for better receiver performance. The ESC Design plans to shift towards 0603 components instead of the current 0402 package. From the firmware team, challenges ahead involve integrating all firmware subsystems with the custom hardware, completing the control system implementation for stable flight, and

finishing USB data population across the remaining desktop application pages. For the software team, structured user testing on both the OSCAR Ground Station application and the OSCAR Docs website is also planned to evaluate usability and accessibility before final delivery.

Learning Summary

Development Standards & Practices Used

Standard Practices Used:

- ISO/IEC/IEEE 12207 – Software Life Cycle Processes
- IEEE Standard 2050-2018 - IEEE Standard for a Real-Time Operating System (RTOS) for Small-Scale Embedded Systems
- IEEE 1923.1-2021 - Recommended Practice for Near-Field Antenna Measurements

Other Standard Practices Considered:

- ISO/IEC/IEEE 26515:2011 - Developing User Documentation in an Agile Environment

Summary of Requirements

- Steady level flight for at least 10 seconds
- 99% USB communication success rate between the drone and the user
- Desktop Application that receives, displays, and updates information from the drone
- Open source documentation that is simple to navigate
- Access to an off the shelf controller (RadioMaster using ExpressLRS)
- Electronics fit on custom 5 inch drone frame
- Thrust to weight ratio provides enough thrust to fly
- Visually appealing and intuitive user interface
- The drone shall maintain a stable hover in outdoor conditions

Applicable Courses from Iowa State University Curriculum

- COM S 3090 - Software Development Practices
- COM S 2280 - Introduction to Data Structures
- COM S 3270 - Advanced Programming Techniques
- COMS 3360 - Introduction to Computer Graphics
- COMS 1270 - Introduction to Computer Programming
- CPR E 1850 - Introduction to Computer Engineering and Problem Solving
- CPR E/EE 2320 - Professionalism and Ethics
- CPR E 2880 - Embedded Systems I: Introduction
- CPR E 3880 - Mobile Platforms and Applications
- CPR E 4880 - Embedded Systems Design
- CPR E 4890 - Computer Networking and Data Communications
- CPR E 3080 - Operating Systems
- EE 2010 - Electric Circuits
- EE 2300 - Electronic Circuits and Systems
- EE 3330 - Electronic Systems Design
- EE 4140/5140 - Microwave Engineering
- ENGL 3140 - Technical Communications
- MATH 1650 - Calculus I
- MATH 1660 - Calculus II
- MATH 2650 - Calculus III
- MATH 2670 - Differential Equations and Laplace Transforms
- PHYS 2310 - Classical Physics 1
- SE 3190 - Construction of User Interfaces
- SE 3390 - Software Architecture and Design
- SE 3170 - Introduction to Software Testing
- SE 4170 - Software Testing

New Skills/Knowledge acquired that was not taught in courses

- Communication associated with being in a team
- How to implement a binary packet protocol with structured headers, length fields, and checksums
- How to design and implement a dual COM port communication architecture
- How to perform COM port scanning and identity verification via handshake sequences
- How to bridge native C libraries into a JavaScript runtime environment
- How to manage concurrent data streams without blocking UI rendering
- How to parse and display live telemetry data from embedded hardware in real time
- How to map quaternion orientation data to a 3D coordinate system for visualization
- How to calculate quaternions
- How to structure a large Electron application using a layered architecture pattern
- How to use IPC (Inter-Process Communication) to safely separate frontend and backend processes in Electron
- How to use callback mechanisms to forward streaming data from a backend thread to a frontend UI
- How to build and host an open-source documentation website using GitHub Pages
- How to package and distribute a desktop application for Windows end users

Table of Contents

List of Figures 8
List of Tables 9
Definitions 10
1 Introduction 13
1.1 Problem Statement 13
1.2 Intended Users 13
2 Requirements, Constraints, And Standards 14
2.1 Requirements & Constraints 14
2.2 Engineering Standards 14
3 Project Plan 17
3.1 Project Management/Tracking Procedures 17
3.2 Task Decomposition 17
3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria 20
3.4 Project Timeline/Schedule 21
3.5 Risks and Risk Management/Mitigation 23
3.6 Personnel Effort Requirements 25
3.7 Other Resource Requirements 26
4 Design 27
4.1 Design Context 27
4.1.1 Broader Context 27
4.1.2 Prior Work/Solutions 28
4.1.3 Technical Complexity 29
4.2 Design Exploration 31
4.2.1 Design Decisions 31
4.2.2 Ideation 32
4.2.3 Decision-Making and Trade-Off 33
4.3 Proposed Design 35
4.3.1 Overview 35
4.3.2 Detailed Design and Visual(s) 36
4.3.3 Functionality 48
4.3.4 Areas of Concern and Development 49
4.4 Technology Considerations 50
4.5 Design Analysis 54
5 Testing 57
5.1 Unit Testing 57
5.2 Interface Testing 59
5.3 Integration Testing 60
5.4 System Testing 61
5.5 Regression Testing 61
5.6 Acceptance Testing 61

5.7 User Testing 62
5.8 Results 62
6 Implementation 64
7 Ethics and Professional Responsibility 70
7.1 Areas of Professional Responsibility/Codes of Ethics 70
7.2 Four Principles 71
7.3 Virtues 72
8 Closing Material 76
8.1 Conclusion 76
8.2 References 77
8.3 Appendices 79
9 Team 80
9.6 Team Contract 82

List of Figures

Figure 1. Hardware Gantt Chart Timeline
Figure 2. Firmware Gantt Chart Timeline
Figure 3. Firmware To Software Gantt Chart Timeline
Figure 4. Software Gantt Chart Timeline
Figure 5. High Level project diagram.
Figure 6. High Level diagram of the flight controller.
Figure 7. High level diagram of the electronic speed controller
Figure 8. USB protocol flowchart
Figure 9. IMU ICM-40609 initialization flowchart.
Figure 10. IMU BNO055 Initialization sequence to use the magnetometer only.
Figure 11. Barometer ENS220 initialization flowchart.
Figure 12. Desktop application architecture overview.
Figure 13. Frontend UI flowchart.
Figure 14. Inter-Process Communication flowchart.
Figure 15. Backend flowchart.
Figure 16. OSCAR use-case example flowchart.
Figure 17. 3D printed frame for the drone.
Figure 18. 3-D rendering and layout of the flight controller.
Figure 19. 3-D rendering of the electronic speed controller.
Figure 20. Layout of the electronic speed controller.
Figure 21. 3-D rendering of the GNSS board.
Figure 22. Layout of the GNSS board.

List of Tables

Table 1. Personal Effort Requirements
Table 2. Impact analysis across public, global, environmental, and economic domains.
Table 3. Mapping of system subsystems to engineering principles and their technical justification
Table 4. Overview of embedded communication protocols.
Table 5. Evaluation of embedded communication interfaces.
Table 6. Weighted comparison of cross-platform development frameworks.
Table 7. Embedded technologies and their roles within the flight controller system architecture.
Table 8. Software architecture and technology stack for the UAV ground control system.
Table 9. IEEE ethics principles mapped to project implementation and engineering practices.
Table 10. IEEE Code of Ethics principles applied to UAV system development and engineering practice.

Definitions

Hardware Definitions:

- **Back EMF - Back Electromotive Force:** The voltage generated by an electric motor as it spins, which pushes back against the driving current. Sensorless ESCs read this voltage to determine the precise position of the motor's rotor.
- **CAN - Controller Area Network:** A robust, differential communication bus standard.
- **DMM - Digital Multimeter:** An electronic measuring instrument used during hardware testing and debugging to measure voltage, current, and resistance.
- **EEPROM - Electrically Erasable Programmable Read-Only Memory:** A type of non-volatile memory used to store small amounts of data (like configuration settings or calibration data) that must be retained even when the device loses power.
- **EMI - Electromagnetic Interference:** A disturbance generated by an external source (such as high-current motor wires) that affects electrical circuits.
- **ESC - Electronic Speed Controller:** An electronic circuit responsible for regulating and controlling the spinning of an electric motor based on signals received from the flight controller.
- **ESD - Electrostatic Discharge:** The sudden, momentary flow of electricity between two differently charged objects.
- **FC - Flight Controller:** The central processing unit of a drone or aircraft. It reads data from onboard sensors (like the IMU and GPS) and user inputs, then calculates and sends signals to the ESCs to maintain stable flight.
- **GPS/GNSS - Global Positioning System / Global Navigation Satellite System:** Satellite-based navigation systems used to determine the device's geo-spatial positioning.
- **IMU - Inertial Measurement Unit:** An electronic component containing accelerometers and gyroscopes that measures the device's specific angular velocity and linear acceleration.
- **LDO - Low Drop Out (Regulator):** A DC linear voltage regulator that can maintain a steady output voltage even when the supply voltage is very close to the desired output voltage.
- **LNA** - Low Noise Amplifier: A component to amplify small signals with a small Noise figure.
- **LGA - Land Grid Array:** A surface-mount packaging technology for integrated circuits (commonly used for IMUs and sensors) where the connection points are a grid of flat metal pads on the bottom of the chip.
- **LiPo - Lithium Polymer:** A type of rechargeable battery heavily favored in RC and drone applications due to its lightweight form factor, high energy capacity, and ability to deliver very high discharge currents.
- **MCU - Microcontroller Unit:** A compact integrated circuit designed to govern a specific operation in an embedded system. It acts as the "brain" of the board, containing a processor, memory, and programmable input/output peripherals.
- **MOSFET - Metal-Oxide-Semiconductor Field-Effect Transistor:** A type of transistor widely used for amplifying or switching electrical signals and power. In this context, they are heavily used in ESCs to rapidly switch power to the motors.
- **Open Source PCB Design Software:** A free, professional-grade electronic design automation (EDA) software suite used to create electrical schematics and translate them into Printed Circuit Board (PCB) layouts.

- **PCB - Printed Circuit Board**: A board made of insulating material (usually FR4) with conductive copper pathways etched onto it, used to mechanically support and electrically connect electronic components.
- **PID - Proportional Integral Derivative:** A control loop feedback mechanism widely used in flight controllers. It continuously calculates the error between a desired setpoint and a measured variable, applying corrections to keep the system stable.
- **RF - Radio Frequency:** Radio frequency is the oscillation rate of an alternating electromagnetic field in the frequency range at which energy from an oscillating current can radiate off a conductor into space as radio waves.
- **RPM - Rotations Per Minute:** A unit of frequency used to measure the rotational speed of a mechanical component, such as a motor's shaft.
- **SWD - Serial Wire Debug:** A two-wire hardware interface protocol used to program, flash, and debug ARM-based microcontrollers.
- **USB - Universal Serial Bus:** A standard interface for connecting external devices to a computer. In this design, it is used for configuring the MCU, flashing firmware, and serial communication.

Firmware Definitions:

- **COM port - Communications port:** A physical or virtual serial interface on a computer used to communicate with peripheral devices.
- **CRC (CRC-16/CCITT checksum) - CRC = Cyclic Redundancy Check, CCITT = International Telegraph and Telephone Consultative Committee:** An error detection algorithm.
- **CPU - Central Processing Unit:** The primary computing core located inside the Microcontroller Unit (MCU) which executes the firmware instructions.
- **CubeIDE :** The platform we use to develop embedded c code and deploy it to the board.
- **DSHOT - Digital Shot:** Communication protocol made to send throttle values and request telemetry data from electronic speed controllers.
- **ExpressLRS/ELRS:** A low-latency, long-range wireless communication protocol used for transmitting control signals between a controller and receiver (such as the EP1 module), built on LoRa technology.
- **H7 nucleo board:** ST Microelectronics development board equipped with test hardware as a platform to prototype embedded code.
- **I2C - Inter-Integrated Circuit:** A type of protocol used to communicate with peripherals.
- **LoRa - Long Range:** A wireless modulation technique designed for long-distance, low-power communication over radio frequencies.
- **OS/RTOS - Realtime Operating System:** Manages the time allocation of tasks in the code.
- **PWM - Pulse Width Modulation:** A signal with varying high and low digital signals.
- **SPI - Serial Peripheral Interface:** A type of protocol used to communicate with peripherals.
- **ST-link:** A USB connector that connects to the board and allows us to debug code in real time.
- **UART - Universal Asynchronous Receiver-Transmitter:** A type of protocol used to communicate with peripherals.

Software Definitions:

- **API - Application Programming Interface:** A set of rules and protocols that allow different software applications to communicate and exchange data with each other. It enables one program to request services or information from another without needing to know the internal workings of that application.
- **CSS - Cascading Style Sheets:** A style sheet language used to assist with the presentation and layout of documents written in markup languages. Most commonly used with HTML.
- **Electron:** An open-source framework that enables developers to build desktop applications using web technologies.
- **FPS - Frames per second:** The frequency at which consecutive images (also known as frames) are captured or displayed. This indicates how many unique images appear on a screen every second in order to create motion. Commonly used to measure video smoothness or gaming performance.
- **GUI/UI - Graphical User Interface / User Interface:** The visual and interactive layer of a software application.
- **HTML - Hypertext Markup Language:** The standard markup language that is used when creating and structuring web pages.
- **IDE - Integrated Development Environment:** Software that combines essential coding tools (code editor, debugger, compiler/interpreter) into one application for the betterment of developer life.
- **IPC - Inter-Process Communication:** A mechanism that allows processes to communicate and share data with each other while they are running.
- **JSON - JavaScript Object Notation:** A lightweight, text-based format that is used to store and exchange data.
- **libserialport:** An open-source C library that handles OS-specific details for accessing serial ports and enabling low-level serial communication.
- **Native Addon:** Modules written in C or C++ that can be loaded into Node.js as if they were standard JavaScript modules.
- **Node-API:** Toolkit built into Node.js used to build native addons. Very commonly used with Electron applications.
- **Node.js:** An open-source JavaScript based runtime environment allowing developers to run JavaScript code outside of a web browser.
- **PC - Personal Computer:** A microcomputer designed for individual use in homes, offices, or schools.

1 Introduction

1.1 Problem Statement

Drones have become one of the most accessible ways to explore embedded systems and control theory, yet the people most eager to learn from them, including students, hobbyists, and researchers, are often locked out of how they actually work. Commercial drones ship as sealed products, and even the open source corners of the drone world are scattered. Open firmware typically runs on closed PCBs, and the rare open hardware design usually lacks the matching firmware or software needed to make it fly. Anyone curious enough to dig deeper ends up piecing together documentation from different projects that were never designed to work together, with no single reference to learn from.

OSCAR addresses this by releasing a complete drone design covering hardware, firmware, and software. Developed together as a single coherent reference. Every schematic, PCB layout, parts list, and line of code is published openly, so anyone with the curiosity to learn can study, modify, and rebuild the entire system end to end.

1.2 Intended Users

We have identified three groups of people who stand to benefit most from OSCAR, and understanding these users is what drives the decisions we make in this project.

The primary demographic is researchers. This is a graduate or PhD student who is motivated by publishing and making progress in their field of research. They do not have time for rebuilding a drone platform from scratch every time they wish to run an experiment. They need something sufficiently configurable and easily adjustable for them to integrate their own research component into the drone. OSCAR's fully open design and software gives them that, so they can spend their time on actual research instead of reinventing the wheel.

The second group is the hobbyist. This is someone whose age can range from high school to adulthood. These hobbyists enjoy spending their time building drones and exhibiting their equipment alongside the skill/art of flying them. They are most concerned with getting the best product available or the newest equipment related to the hobby. OSCAR would allow the hobbyist to easily add their own designs from either hardware or software into a drone they want to design for personal use.

The third is an engineering student; a person who wants to better understand a system design and wants access to how things work. They have outlying motivation from peers and instructors to be in the constant pursuit of knowledge as well as an internal drive to learn more about topics they find interesting. Students may not have the breadth or depth of knowledge needed to parse together the many different drone resources online. OSCAR would provide the resources needed to understand drone design and build their own.

All three of these users share the same core problem: the information and access they need is not readily available for them. OSCAR strives to change this fact and become a central hub for custom drone design and alteration.

2 Requirements, Constraints, And Standards

2.1 Requirements & Constraints

- Functional requirements
    - Steady level flight
    - Communication between the drone and the user
    - Desktop Application that receives, displays, and updates information from the drone
- Resource requirements can
    - Open source documentation
    - Access to an off the shelf controller (RadioMaster using ExpressLRS)
- Physical requirements
    - Electronics fit on custom 5 inch drone frame
    - Thrust to weight ratio provides enough thrust to fly
- Aesthetic requirements
    - The user interface should be visually appealing
    - The drone should have all necessary parts attached
- User experiential requirements
    - Intuitive user interface
    - Easily navigable open source documentation
- Environmental requirements
    - The drone shall maintain a stable hover regardless of outside disturbances
    - Operating altitude range (proximity to airport) 100ft height constraint following FAA regulations

2.2 Engineering Standards

Engineering standards are important to create cohesion across the field. This allows for modular designs to support incorporating other pieces of technology. Another important advantage of using common standards is ensuring safe operation of what is being engineered both for the users and general public.

Standards Found

The software team found the ISO/IEC/IEEE 12207 – Software Life Cycle Processes standard is an international standard defined collaboratively by ISO (International Organization for Standardization), IEC (International Electrotechnical Commission), and IEEE (Institute of Electrical and Electronics Engineers). It defines a common framework for software lifecycle management, from conception, acquisition, development, operation, maintenance, and disposal. The goal of this standard is to provide a way for all parties involved in a software project (developers, acquirers, users, stakeholders, etc.) to communicate effectively during the lifecycle of the software system and ensure all parties are involved and satisfied.

The firmware team found the IEEE Standard 2050-2018 addresses real-time operating systems and their implementation on resource-constrained microprocessors. The standard outlines processes for maintaining the optimization and speed of an OS on lightweight hardware. This standard's goal is to achieve portability, reusability, and optimization for resource-constrained MCUs. This enables

the wide-ranging use of real-time operating systems across a wide variety of microcontroller units and projects.

The hardware team found IEEE 1923.1-2021, Recommended Practice for Near-Field Antenna Measurements. This standard is a recommended practice that establishes standardized procedures for measuring antenna performance using near-field measurement techniques, where the antenna is characterized in close proximity rather than at the large distances required for traditional far-field testing. The standard is intended to ensure consistency and accuracy across different test facilities and methodologies by defining acceptable measurement setups. Antenna metrics such as gain and radiation pattern, and characterization procedures for polarization behavior. This is particularly important for the usage of a patch antenna for this project for our GPS receiver.

Relevance to Project

After reviewing some of the technical details of the ISO/IEC/IEEE 12207 – Software Life Cycle Processes Standard, we have determined that it is extremely relevant to our project. As part of this project, we will make all our work open source. Since a large part of this project involves creating software and firmware, we need to ensure everything is easy to follow, repeatable, and organized so that anyone looking at our project can understand at least where everything is located. Since we are under a time constraint, we also need to ensure we have a well-defined lifecycle scope so we can finish everything on time.

After reviewing IEEE 2050-2018, we have determined that it is relevant to our project. We need to be able to validate the real-time operating system we are using. In addition, we need to ensure that changes to the operating system's implementation remain in accordance with this standard. Our project is open source and intended to run on custom hardware. Meeting this standard will help keep our project effective as an open source implementation.

IEEE 1923.1 - 2021 is directly relevant to this project because the OSCAR quadcopter relies on a GPS module with a passive inset-fed microstrip patch antenna designed to receive signals at 1.575 GHz on the GPS L1 band. For the antenna to function correctly within the system, its radiation pattern, gain, and polarization characteristics must be verified against design targets before integration. This standard provided exactly that framework, it defined how near-field measurements should be conducted to extract those parameters in a controlled way.

Reflection Of Standards

The Firmware team chose the IEEE 2050-2018 standard. It differs from the one selected by Software because it focuses on how they are developing their operating system, whereas the one we selected focuses on how our team develops the project as a whole and takes a much more managerial stance than the development-focused approach that Firmware's chosen standard takes.

The Hardware team chose the IEEE 1923.1-2021 standard. This standard differs from the one selected by Firmware because it addresses hardware measurements, particularly near-field antenna characterization. While Firmware focuses on real-time OSs and resource-constrained microcontrollers, the Hardware standard concerns physical signal integrity and component layout.

The Software team chose the ISO/IEC/IEEE 12207 – Software Life Cycle Processes Standard. This standard particularly differs from the hardware selected standard; this measures how a system level

software system is handled and documented over its life cycle.Compared to IEEE 1923.1 - 2021 which is very technical in just antenna design and validation, the software standard concerns an entire system level is maintained.

Modifications to Project in Order to Incorporate Standards

We want to ensure we take the Software Lifecycle standard into account when developing our project documentation and the documentation website. According to what we know about this standard, there are important processes for documentation and traceability. As we develop our documentation for this open-source project, we want to ensure it enables seamless implementation and learning for any user. Further review of this standard has pointed us to the ISO/IEC/IEEE 26515:2011(en) Systems and software engineering — Developing User Documentation in an Agile Environment standard. This will likely be a useful standard for us to consider as well. (https://www.iso.org/obp/ui/en/#iso:std:iso-iec-ieee:26515:ed-1:v2:en)

We plan to validate that our real-time operating system implementation meets the requirements outlined in IEEE 2050-2018. This includes optimizing scheduling, resource usage, and timing guarantees. Any changes to the RTOS or its configuration will be tested to ensure compliance with the standard. This ensures that our firmware is reliable, portable across microcontrollers, and suitable for real-time drone control.

To incorporate IEEE 1923.1-2021 into the project, the team intends to validate the GPS patch antenna's performance using measurement equipment available through the Microwave Engineering department at Iowa State University. Specifically, a Vector Network Analyzer (VNA) will be used to measure the antenna's input impedance and return loss (S11), confirming that the antenna is properly matched at 1.575 GHz before integration into the system.

3 Project Plan

3.1 Project Management/Tracking Procedures

For our senior design project, we are using a hybrid approach that combines elements of both waterfall and agile methodologies. Our process follows a waterfall structure in that we prioritize and develop the system in stages, starting with the most critical components before moving on to less essential features. At the same time, we incorporate agile practices by working in short, deadline-driven cycles where each team member is responsible for implementing, testing, and debugging their assigned features. This allows us to maintain overall structure while still adapting and making continuous progress throughout development.

Our team will be tracking the progress made throughout this course using GitLab, Discord and Google Sheets. GitLab will be used to track changes and contributions to the project. Google Sheets will be used to track the actual tasks and their progression. It will also track who is assigned to which task, its start date, end date, and priority. Discord will be the main form of communication. It will be where we ask each other questions, give each other updates/feedback and help each other with problems when we are not together.

3.2 Task Decomposition

To decompose our tasks we began by separating the tasks into 3 groups associated with discipline. Those groups are Hardware, Firmware, and Software tasks.

Hardware:

- Flight Controller Design
    - USB 2.0 circuity
    - Microcontroller circuity
    - Voltage regulators
    - IMU - Inertial Measurement Unit circuity
    - Barometer circuity
    - External Oscillator circuity
    - ESD ProtectionBoard layout
    - Bill of Materials generation
    - Design Justification Document
    - First revision order placement
    - First revision board assembly
    - First revision board functionality testing
    - First revision board benchmarking
- GNSS Board
    - Magnetometer
    - GNSS Receiver
    - Patch Antenna
    - Board layout
    - Bill of Materials generation
    - Design Justification Document
    - First revision order placement

    - First revision board assembly
    - First revision board functionality testing
    - First revision board
- Electronic Speed Controller
    - Microcontroller circuit
    - Gate driver circuit
    - Gate driver voltage regulation circuit
    - Microcontroller voltage regulation circuit
    - Transient voltage suppression
    - Board layout
    - Bill of Materials generation
    - Design Justification Document
    - First revision order placement
    - First revision board assembly
    - First revision board functionality testing
    - First revision board benchmarking

Firmware:

- DSHOT Communication Protocol
    - Flight controller implementation
    - ESC DSHOT receive
        - Establish an arming state machine
        - Establish motor driving logic
    - Establish customizations through USB communication
- USB Communication
    - Establish a Virtual COM port
    - Establish data packet organization
    - Establish receive and transmit logic
    - Establish CRC error handling
    - Establish continuous data streaming for Quaternions
- Barometer
    - I2C Communication
    - Configuration
- IMU
    - SPI Communication
    - Accelerometer Setup
    - Gyroscope Setup
- GPS
    - UART Communication
    - Configuration
- Magnetometer
    - I2C Communication
    - Configuration
- Express LRS Protocol
    - UART Communication
    - Setup

    - Configuration
- Integrate FREERTOS
    - Integrate the libraries for FREERTOS
    - Create OS tasks for required functionality
- Controls
    - Quaternion building
    - Quaternion Application
    - State Space Modeling

Software:

- USB Communication
    - Implement CRC error handling
    - Search for and connect to the correct COM port
    - Set up communication of requests across the frontend and backend of the Desktop Application
    - Implement data streaming through the COM port
    - Implement sending and receiving packets
    - Implement the ability to parse through data packets
    - Write Documentation for USB implementation
- Desktop Application
    - Home Page
    - Setup and Ports Page
        - Gyroscope Preview
    - Configuration Page
    - Power and Battery Page
    - Motors
    - PID Tuning
        - PID Profile Tab
        - Rateprofile Tab
        - Filter Settings Tab
    - GPS Page
    - Receiver Page
    - Write documentation for Desktop Application Implementations
    - Create an executable file to install the Desktop Application
    - Visually pleasing user interface
- Open Source Documentation Website and Repository
    - Easily navigable documentation
    - Visually pleasing user interface
    - All project files are accessible through the repository

3.3 Project Proposed Milestones, Metrics, and Evaluation Criteria

Key Milestones:

Some milestones for this project would include having a functional Flight controller that can receive and send data over USB, read clean data from its sensors, and send the commands needed to fly the quadcopter to the ESC. The GNSS board needs to receive GPS, BeiDou, and GLONASS signals via a passive patch antenna. The data received from the GNSS module will be sent to the Flight controller to assist in stable level flight. The ESC needs to receive DSHOT data from the flight controller, achieve accurate motor rpm, read clean ESC sensor data, and maintain a low enough temperature to prevent component and board damage. There will also need to be a downloadable Desktop Application that can configure drone settings via USB communication with the drone. Finally, there will be an open-source repository on GitHub containing all project files, along with a GitHub Pages website that will host all project documentation and instructions.

Progress Measurement:

To measure progress from the flight controller, GNSS Board, and ESC from a hardware perspective, the following will be measured. Some general testing for continuity testing to ensure there are no short circuits on our board before powering up. Then performing isolated testing on the voltage regulation. Once those tests are passed, we will then ensure communication is occurring over SWD to ensure MCU is turning on through CUBEIde. Once MCU communication is occurring, then trying to flash over USB will be the next test if hardware on the USB is correct. GNSS Board is an RF Board and requires special attention since its operating at 1.575GHz thus using a VNA will be used to measure our S11 performance. ESC will be measured based on its step response to the motors, in addition to thermal performance.

To measure progress from a Firmware perspective, the following scheme will be used. Starting progress will be directly tied to research done on datasheets and implementation material. This research is vital for most tasks as programming the implementations requires a detailed approach. Progress can be determined as halfway once an implementation has been and appears to work. A task can be determined complete once the implementation has been tested to determine 80% consistent reliability.

To measure progress from the software team's perspective, we will use the following: The USB Communication milestone will be considered complete if data can be sent and received from the flight controller to the desktop application with an accuracy of 95%, within a time frame of 100ms. To measure the Desktop Application milestone, we will need to have a downloadable application that has a user friendly interface and triggers USB communication based on user input. It will also need to include an operational webpage for each milestone listed (Home Page, Setup and Ports Page, Configuration Page, Power and Battery Page, Motors Page, PID Tuning Page, GPS Page, Receiver Page). The open source documentation website and repository will be considered complete if every major milestone from each sub-team has its associated documentation included on the website and all project files are in the repository and can be accessed by the general public by the end of the project.

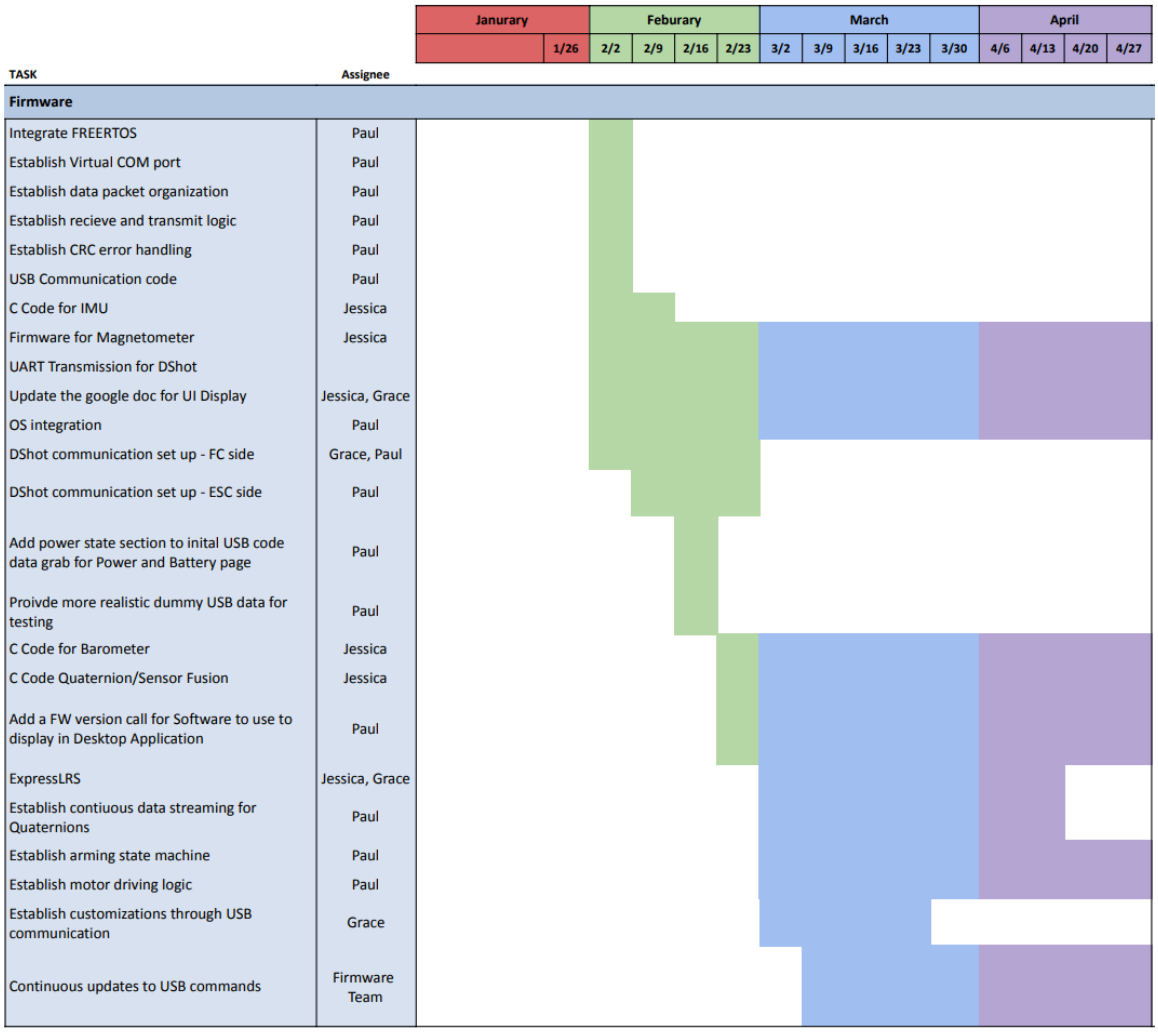
3.4 Project Timeline/Schedule

OSCAR Timeline

		Janurary		Feburary				March					April			
			1/26	2/2	2/9	2/16	2/23	3/2	3/9	3/16	3/23	3/30	4/6	4/13	4/20	4/27
TASK	Assignee															
Hardware																
ESC Rev 1	Collin															
ESC Microcontroller circuit	Collin															
ESC Gate driver circuit	Collin															
Flight Controller Rev 1	Matthew															
FC external ocillator circuit	Matthew															
FC voltage regulators	Matthew															
ESC Gate driver voltage regulation circuit	Collin															
FC Microcontroller circuit	Matthew															
FC USB circuit	Matthew															
ESC Microcontroller voltage regulation circuit	Collin															
ESC Transient voltage supression	Collin															
FC barmometer circuit	Matthew															
FC IMU circuit	Matthew															
GPS board	Matthew															
FC ESD protection circuit	Matthew															
Layout Magnatometer	Matthew															
ESC Board layout	Collin															
ESC Design justification document	Collin															
FC design justification document	Matthew															
ESC bill of materials generation	Collin															
ESC first revision order placement	Collin															
FC bill of materials generation	Matthew															
ESC first revision board assembly	Collin															
ESC first revision functionality testing	Collin															
FC first revision board assembly	Matthew															
FC first revistion board functionality testing	Matthew															

Figure 1. Hardware Gantt Chart Timeline

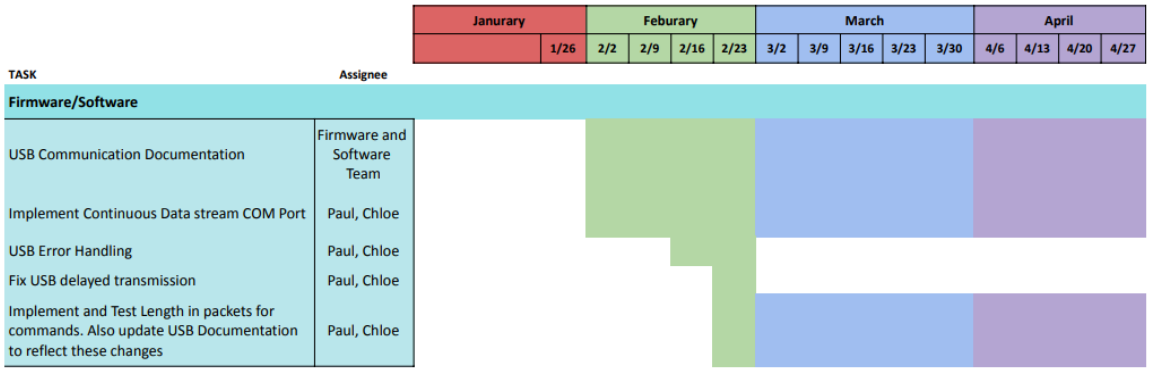
OSCAR Timeline



TASK	Assignee
Firmware	
Integrate FREERTOS	Paul
Establish Virtual COM port	Paul
Establish data packet organization	Paul
Establish recieve and transmit logic	Paul
Establish CRC error handling	Paul
USB Communication code	Paul
C Code for IMU	Jessica
Firmware for Magnetometer	Jessica
UART Transmission for DShot	
Update the google doc for UI Display	Jessica, Grace
OS integration	Paul
DShot communication set up - FC side	Grace, Paul
DShot communication set up - ESC side	Paul
Add power state section to inital USB code data grab for Power and Battery page	Paul
Proivde more realistic dummy USB data for testing	Paul
C Code for Barometer	Jessica
C Code Quaternion/Sensor Fusion	Jessica
Add a FW version call for Software to use to display in Desktop Application	Paul
ExpressLRS	Jessica, Grace
Establish contiuous data streaming for Quaternions	Paul
Establish arming state machine	Paul
Establish motor driving logic	Paul
Establish customizations through USB communication	Grace
Continuous updates to USB commands	Firmware Team

Figure 2. Firmware Gantt Chart Timeline

OSCAR Timeline



TASK	Assignee
Firmware/Software	
USB Communication Documentation	Firmware and Software Team
Implement Continuous Data stream COM Port	Paul, Chloe
USB Error Handling	Paul, Chloe
Fix USB delayed transmission	Paul, Chloe
Implement and Test Length in packets for commands. Also update USB Documentation to reflect these changes	Paul, Chloe

Figure 3. Firmware To Software Gantt Chart Timeline

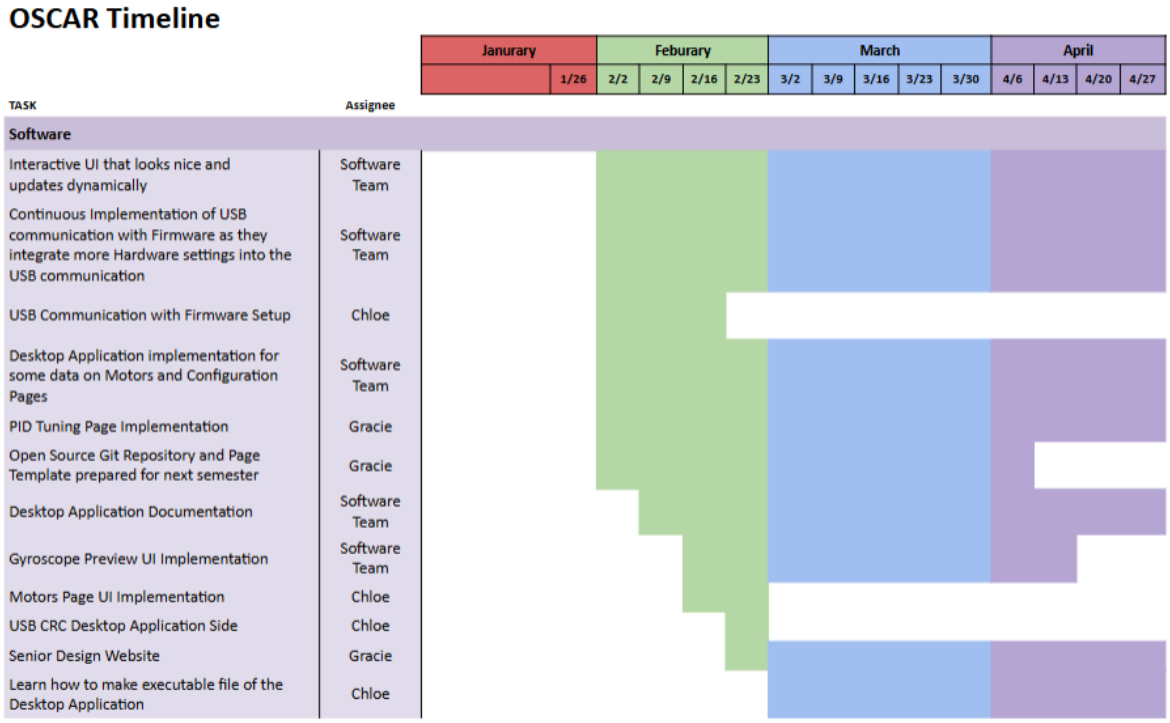


Figure 4. Software Gantt Chart Timeline

3.5 Risks and Risk Management/Mitigation

- Flight Controller Design
    - For any power circuitry the risk of failure is 5% since this design has been made successfully before by team member Matthew Smith
    - For the USB circuitry, the risk is 10% as this has also already been made before
    - For the peripheral sensors, we have used development boards to mitigate integration risks already (10% per sensor).
- GNSS Board
    - Antenna gain dBi not being high enough to receive GNSS signals from satellites is about a 15% failure rate
- Electronic Speed Controller
    - For all power circuits, we will implement a breakpoint in the PCB to allow for testing each circuit independently. This reduces the risk of failure to about 20%.
- DSHOT Communication Protocol
    - The main risk for DShot is that there is a 0.1 chance of not meeting timing requirements. If this were to happen, the clocking equations would need to be rewritten or a new microcontroller would need to be purchased.
- USB Communication
    - A risk associated with the usb communication is loss of connection with the host device or a loss of data, this risk could be around 0.5. The mitigation for this is error checking algorithms on both the host pc and firmware. The algorithms will do constant checks on COM port status to notify the user of a disconnect. The

firmware and software will also have CRC checks one either side to detect data loss. If data loss is found commands will be resent.

- Barometer
    - Could have drift (0.1) , or I2C instability (0.05%). In order to mitigate this risk we would form a better filtering algorithm or sensor fusion to fix drift. Adding pull-up resistors would help with the I2C instability. Given the lack of alternatives, we would cut our losses and get rid of it. Not mission-critical.
- IMU
    - We have a risk that the data is not updated enough and stale data is being transmitted (0.25). Another issue is that our SPI communication will not be working (0.25). These risks will be mitigated by researching these issues and debugging to the best of our ability. After that, we might switch to an off the shelf breakout board such as the BNO055.
- GPS
    - Could have drift (0.1) , or connection instability (0.05%). In order to mitigate this risk we would form a better filtering algorithm or sensor fusion to fix drift. Given the lack of alternatives, we would cut our losses and get rid of them. Not mission-critical.
- Magnetometer
    - Could have drift (0.1) , or I2C instability (0.05%). In order to mitigate this risk we would form a better filtering algorithm or sensor fusion to fix drift. Adding pull-up resistors would help with the I2C instability. If It does not work we would switch to an off the shelf alternative because it is mission-critical.
- Express LRS Protocol
    - We have a communication risk that the controller and receiver could not communicate (.1). To fix this we could put the controller closer to the receiver or flash both the controller and receiver with identical versions of the firmware.
    - We have a 10 percent risk that the controller could send invalid throttle values. To mitigate this we will check the throttle values before sending them and throw out invalid data.
- Integrate FREERTOS
    - Some risks would be the scheduler not giving enough time to time sensitive tasks. The risk of this would be about 0.2 as testing would alleviate these problems. A mitigation approach would be to include state machines that alter the state of the drone if certain timing requirements aren't being met.
- Desktop Application
    - The risks associated with the Desktop Application would be writing code with security vulnerabilities that could cause harm either during app installation or while the app is running. To mitigate this, the tool being used to create the executable file checks for these types of vulnerabilities and throws errors when they are caught.

3.6 Personnel Effort Requirements

Table 1. Personal Effort Requirements

Task	Team	Estimated Hours	# People	Total Person-Hours	Notes
USB Error Handling	Firmware/Software	8	2	16	Debugging + testing CRC
Fix USB Delayed Transmission	Firmware/Software	6	2	12	Packet length + validation
USB Communication Documentation	Firmware/Software	2/week	3	60	Ongoing over ~10 weeks
DShot Communication (ESC Side)	Firmware	20	1	20	Signal handling + logic
DShot Communication (FC Side)	Firmware	20	2	40	Output + protocol timing
Express LRS Integration	Firmware	25	2	50	Research + implementation
IMU C Code	Firmware	15	1	15	SPI + validation
Barometer Code	Firmware	12	1	12	Debugging + calibration
Sensor Fusion (Quaternion)	Firmware	25	1	25	Algorithm + validation
Magnetometer Integration	Firmware	10	1	10	Raw data + tuning
UI Development	Software	4/week	2	80	Iterative over semester
USB Communication (Software Side)	Software	25	1	25	Initial Integration + testing
Desktop App Features (Motors,	Software	4/week	2	60	UI + backend logic + USB

PID, etc.) & connecting to					communicati on updates
Task	**Team**	**Estimated Hours**	**# People**	**Total Person-Hours**	**Notes**
Gyroscope UI Implementation	Software	12	2	24	Visualization + data hookup
Hardware (ESC Board, Layout, etc.)	Hardware	30	2	60	Design + testing

3.7 Other Resource Requirements

Our main other resource is simply time. Each task for this project takes a significant amount of time for designing, implementing, testing and debugging the system. As well as creating clear communication and documentation for each step of the project. Other important resources include access to development tools and software, lab space and equipment for testing, and team member expertise and collaboration. These resources are essential to ensure steady progress and successful completion of the project.

4 Design

4.1 Design Context

4.1.1 Broader Context

OSCAR targets the academic community: hobbyists expanding their skills, students exploring topics beyond coursework, and researchers needing a transparent platform for experimentation and publication. Open source efforts in the drone ecosystem are fragmented across domains. Firmware projects like ArduPilot, Betaflight, AM32, and Bluejay are widely adopted, but they all run on flight controller and ESC hardware that is overwhelmingly closed. Schematics, PCB layouts, and bill of materials are rarely published, even for boards marketed as "open." Conversely, the few openly published hardware designs often lack matching firmware or grounded software developed alongside them. A student or researcher wanting to understand a complete embedded drone design must therefore stitch together open firmware running on a black-box PCB, with no unified reference to study. OSCAR addresses this gap by releasing a coherent design across all three domains of hardware, firmware, and software.

Table 2. Impact analysis across public, global, environmental, and economic domains.

Area	Description	Examples
Public health, safety, and welfare	Our project will affect the general health, safety, and welfare in a positive way. It will provide free resources to the public as well as a way to understand the development process of a drone. Education is a main priority when working on this project as everything will be made available to the public so they can learn from it.	Publishing a complete flight controller reference design: schematics, PCB layout, and firmware that students can study to learn embedded UAV development.
Global, cultural, and social	Our project reflects the environment of complete open source design, allowing anyone from different backgrounds to understand the process of designing and implementing a UAV.	Open repositories and documentation allow anywhere in the world to reproduce and adapt the design without institutional access or licensing barriers.
Environmental	Our project has environmental impact through the usage of batteries, hazardous metals on the PCBs, and electricity to support the host computer. These uses of electricity help to contribute to the burning of fossil fuels as society has not fully migrated to sustainable resources. The manufacturing of our drone's hardware also requires cooling resources in the factories that produce the components.	Energy consumed charging LiPo batteries during development and flight, plus the raw materials — rare-earth magnets, copper, lead, and refined silicon — required to produce the motors, PCBs, and ICs.
Economic	Our project contains an upfront cost, the hardware required to fly has an initial cost that requires users to purchase which is non affordable since our design is not made for mass manufacturability, but instead on	Upfront purchase of ICs, frame material, motors, and supporting electronics required to fly

	complete open source documentation to better understand the whole system.	

4.1.2 Prior Work/Solutions

Hardware/Firmware:

Homemade drone builds are common, but well documented reference designs spanning both hardware and firmware are rare. On the flight controller side, open source firmware projects like ArduPilot, PX4, and Betaflight were referenced for peripheral handling, but these stacks target predefined hardware and are typically treated as “black boxes” by users who flash them onto closed commercial boards. The flight controller PCBs themselves (Holybro, Matek, SpeedyBee) are overwhelmingly proprietary, with no published schematics, layouts, or BOMs, and the few open FC designs available tend to be outdated or lack design rationale.

The ESC side follows the same pattern: AM32 is an open-source ESC firmware that was referenced for pin mapping and peripheral compatibility, and OSCAR's ESC hardware is designed to support both our custom firmware and AM32. No widely available open electronic speed controller has its design process detailed and documented. OSCAR fills this gap by publishing complete FC and ESC PCB designs with documented decisions for each consideration, paired with firmware that works on both our boards and the existing open source ecosystem.

Software:

Desktop applications are commonly used with drone systems to configure and monitor flight controllers without controlling the drone. One widely used example is Betaflight Configurator, which allows users to adjust parameters such as PID values, calibrate sensors, and update firmware while the drone is stationary. A similar application is Mission Planner, which provides configuration and monitoring features as well as supporting autonomous mission planning. Prior research highlights the challenges of interacting with embedded drone firmware and the need for accessible configuration tools. Firmware analysis research shows that low-level systems can be difficult to manage without user-friendly interfaces [1], which can lead to confusion for newcomers to the drone world.

Compared to existing tools, Betaflight Configurator offers powerful and useful configuration capabilities but can be complex for new users, while Mission Planner includes many features that may be unnecessary for simple setup and tuning tasks. The proposed software focuses specifically on pre-flight and post-flight configuration as well as system feedback while the drone is stationary. It provides a simplified interface designed specifically for the team's hardware and firmware, making it much easier for newcomers to understand and interact with drone configuration settings. This approach reduces complexity and improves usability, even though it may offer fewer advanced features than larger and more popular platforms.

4.1.3 Technical Complexity

Our project, being inter-disciplinary, has a combination of both breadth and depth of technical complexity. We will explain our project's technical complexities by describing each sub-team's work.

Hardware:

The hardware is the backbone of our drone design, providing the mechanics and electronics needed for flight. Key components include the drone frame, which holds the electronics and motors, and the flight controller, which computes control outputs and manages communication with the motors. Additionally, the electronic speed controller translates inputs from the flight controller into motor commands and connects the hardware to the user interface. Each core system requires firmware programming to function properly. Important subsystems to implement include IMU and barometer control, USB device functionality, and DSHOT communication.

Firmware:

The firmware stands as the middle ground between the hardware and the user interface. Each core functionality has to be programmed in firmware in order for it to be realized. The main subsystems that have to be implemented in firmware include IMU and barometer control, USB device functionality, DSHOT communication with electronic speed controllers, and control systems. Our firmware also utilizes an operating system which adds in a major timing component to all of our implementations.

The IMU and barometer subsystems require proper register configuration and communication protocol setup. These peripherals are established using protocols like I2C and SPI. Each of these protocols has its own set of functionality that differ greatly from the other. The implementations of these require extensive reading of datasheets and much time spent debugging to tune properly.

The USB device functionality requires good understanding of the driver being implemented. The driver in use here is TinyUSB. Through the implementation of the USB functionality our implementation requires understanding of device endpoints, CDC configuration, and extensive time spent building a communication packet structure. The packet structure has to be efficient for using across both firmware and software.

The DSHOT communication setup requires usage of PWM timers to achieve fast and responsive communication. Our requirements require that the DSHOT communication protocol communicate at 600KHz. Achieving this functionality requires great knowledge of PWM signals, clock speeds, and timers. Understanding these topics requires extensive reading through datasheets and computer engineering related topics.

Software:

The proposed OSCAR Ground Station desktop application satisfies the requirement for technical complexity through the integration of low-level hardware interfacing with high-level user interface and graphical rendering.

Table 3. Mapping of system subsystems to engineering principles and their technical justification

Subsystem	Applicable Principles	Technical Justification
Hardware Abstraction Layer (HAL)	Digital Communications & UART over USB	The system utilizes a native C module to bridge the gap between the Node.js runtime and the OS-level COM port drivers. This allows for direct UART communication over a USB bridge, ensuring access to the Flight Controller (FC) hardware that standard high-level APIs cannot provide.
Telemetry Integrity Layer	Information Theory & Error Detection	To ensure data integrity, the design implements a CRC-16/CCITT cyclic redundancy check. This involves mathematical polynomial division of the bitstream to detect burst errors.
Concurrency Management	Multi-threaded Systems & Asynchronous I/O	To maintain a smooth user experience, the system utilizes multi-threading. The communication stack (C-module and Serial I/O) runs independently of the Main and Renderer threads, preventing UI freezing during high-frequency data bursts.
Spatial Visualization	Kinematics & Computer Graphics	The Three.js integration performs real-time 3D rendering. This requires mapping telemetry data (Quaternions) from the drone's IMU to a virtual 3D coordinate system, involving matrix transformations to update the drone's position in the viewport.

Unlike other drone configuration desktop applications, this system directly interfaces with a custom flight controller, allowing the user to become intimately familiar with their own specific design. This high level of customization requires a strict COM port validation and handshake sequence. The application must scan, identify, and verify the identity of the flight controller across multiple simultaneous COM ports. This requires a state-machine approach to ensure the correct baud rates and protocol versions are synchronized before telemetry begins.

The project exceeds standard application complexity by managing a continuous Data Stream. This involves handling raw binary packets arriving via UART and converting C-based binary structures into JSON objects for the JavaScript frontend.

4.2 Design Exploration

4.2.1 Design Decisions

Hardware:

The first major decision was the selection of the STM32H7 as the flight controller MCU. The H7 series provides the clock speed, FPU performance, and peripheral count needed to run tight control loops and communication stacks concurrently, while remaining accessible through free toolchains STM32CubeIDE that align with the project's open source goals. On the ESC side, the hardware team has decided to put two ESC circuits on two identical ESC boards to create the four total ESCs for all four motors. This decision was made as the symmetry of a split board design made duplication of the board for the other two motors rather simple. The option of making four identical boards with a single ESC circuit on each was proposed but was rejected due to the height required to stack four ESC boards vertically. The two board design also reduced BOM from a four board design since we would need fewer voltage regulators.

Another design on the ESC board was the use of 2oz thick copper on the internal and external layers of our PCB. This thicker copper will reduce the resistance of the traces and in turn the total power lost to heat/heat generated. Another design choice was avoiding using thermal reliefs which followed a similar thought process to the 2oz copper. We need as much heat dissipation as possible since the motors pull up to 50A.

Firmware:

A critical design decision firmware made was to select the communication protocol for the IMU. The choice carries domino effects on the entire system because the reliability and speed of the IMU is the starting point of the reliability and speed of our entire system. The PID loop, which we designed to operate at 8KHz, needs fresh data from the IMU at a minimum of 8KHz. A communication protocol that cannot reliably deliver data at this speed will cause stale data or bad data to enter the system which will affect the project's core functionality. It can introduce a bottleneck at the sensor layer that propagates directly into the control loop that degrades the system response. Due to all those reasons, the communication protocol selection was a high-priority decision for the firmware team.

Software:

The most important design decisions made by the software team were to use the Electron open-source framework to create our Desktop Application and compiling a C module into our Electron project that handles all communication between the Desktop Application and the Flight Controller. After many other considerations, our team decided to pick Electron because it is known to be easy to pick up, quick to develop on, and provided a lot of flexibility that would allow us to integrate a wide range of features. Additionally, all team members had prior experience with the languages used in Electron, which allowed us to work efficiently. This choice has positively impacted the project by enabling rapid development and contributing to the overall success, as the team has been able to complete a significant portion of the software.

Another key design decision was to manage all USB communication within a C module compiled into Electron. This approach was chosen after researching how JavaScript handles COM port access

and modification, as using a C module provided greater reliability and control. This decision has also positively impacted the project by giving the team full control over communication and allowing smooth data flow and a stable connection between the Desktop Application and the Flight Controller.

4.2.2 Ideation

Hardware:

For the number of ESC circuits on a single ESC board, the hardware team brainstormed with Dr. Neihart who is experienced in board layout and assembly to identify possible and realistic options for the number of boards we would need. We settled on two per board due to the assembly, skill, and time constraints of our senior design team.

Firmware:

The firmware team identified 5 potential options for the IMU communication protocol and they are shown in the table below. Once the protocols were identified, the features were explored and noted in order for us to have a full understanding of the protocol and what it has to offer the system.

Table 4. Overview of embedded communication protocols.

Communication Protocol	**Features**
Serial Peripheral Interface (SPI)	Synchronous, Full Duplex connections, relies heavily on precise timing, supports clock speeds of 1-20 mHz, no addressing overhead
Inter-Integrated Circuit (I2C)	Synchronous, Half Duplex Connection, Easier to interface with, maximum speed of 400kHz, or 1 MHz, requires address handshake before transaction.
Universal Asynchronous Receiver-Transmitter (UART)	Asynchronous, Full Duplex Connection, no shared clock line, agreed baud rate, simple communication
Controller Area Network (CAN) Bus	Asynchronous, Half Duplex, mulit-node bus protocol. Uses differential signaling for strong noise immunity. Robust communication, which has high overhead.
PDM/Direct Analog Output	No digital protocol, IMU outputs continuous analog voltage or pulse density modulated signal directly. Lowest latency but susceptible to electrical noise and low resolution.

Software:

One of the first major decisions that the Software team had to make was the framework we wanted to develop our desktop application with. The central theme of our decision-making process was developer proficiency – to find a framework that does not have an incredibly steep learning curve so

that we could begin development right away rather than teaching an entire new system to ourselves at the expense of the quality of the product, given our time constraint. Some other qualities we wanted to consider in the framework we chose were helpful documentation availability, programming language familiarity, and efficiency and speed. In our process of deciding on the framework for developing our desktop application, we considered multiple different options: Electron (Selected), Tauri, Flutter, Qt, and .NET

4.2.3 Decision-Making and Trade-Off

Hardware:

The hardware team identified the pros and cons through researching existing designs and cross checking their design capabilities with our own. If the designs were well outside of our assembly capabilities (e.g. lead-less components on both sides of the board) we would rule the design out. We weighed the rest based on likelihood of success, cost of parts, expected time to design, part stock, and lead time.

Firmware:

The firmware team identified the pros and cons through research associated with the characteristics or traits of the protocol. Factors such as wiring simplicity and ease of implementation were considered, but throughput and latency was the highest priority. Any protocol that could not reach the 8KHz requirement was not a viable option for the IMU.

Table 5. Evaluation of embedded communication interfaces.

Protocol	Pros	Cons
SPI	• Fastest throughput low latency • Supported by most IMUs	• Requires more signal lines • More complex to debug • Requires precise timing
I2c	• Simple interface • Easy to implement • Supported by most IMUs	• Half Duplex • Slower max speed • Address Overhead
UART	• Full Duplex • Simple	• Not supported by most IMUs • Asynchronous Timing
CAN	• Good Noise Immunity • Robust Error Detection • Good in noisy environments	• High protocol overhead • Overkill for single sensor
PDM/Analog	• Lowest Latency • No Overhead	• Susceptible to electrical noise • Low Resolution • No Configurability

Following this analysis we eventually selected SPI. It satisfies the 8KHz data rate, requires no additional hardware, and is directly supported by our MCU and most IMUs. We had concerns about the difficulty in connecting with a device via SPI, but speed needed to be our number one priority.

Software:

We developed a Weighted Decision Matrix to assist us in making our decision between the frameworks available:

Table 6. Weighted comparison of cross-platform development frameworks.

Criteria (Weight)	Electron	Tauri	Flutter	Qt	.NET
Developer Proficiency (0.35)	5	2	2	1	3
Documentation Availability (0.25)	5	3	4	5	5
Programming Lang. Familiarity (0.25)	5	2	2	1	3
Efficiency / Speed (0.15)	2	5	4	5	4
Weighted Total	**4.55**	**2.70**	**2.80**	**2.50**	**3.65**

We also developed a simple pros and cons list during the decision-making process:

- Electron (Selected)
  - Pros: Large ecosystem, developer proficiency, widely popular, cross-platform
  - Cons: uses a lot of memory
- Tauri
  - Pros: Smaller binaries, better performance, uses system webview instead of bundling Chrome
  - Cons: Smaller ecosystem, more setup, requires Rust knowledge
- Flutter
  - Pros: Single codebase, consistent UI, good performance
  - Cons: Uses Dart
- Qt
  - Pros: Mature, fast, native look, used in real professional software
  - Cons: Licensing complexity, C++ difficulties
- .NET
  - Pros: Native, well-supported
  - Cons: Windows-specific

The Weighted Decision Matrix quantified our trade-off analysis. While Qt and Tauri outperformed Electron in the Efficiency / Speed category, their scores in Developer Proficiency and Language Familiarity were significantly lower. Given the project's timeline, the risk of a learning curve for Rust or C++ was deemed greater than the risk of high memory usage. Therefore, Electron emerged as the

optimal choice with a weighted score of 4.55, ensuring that we could immediately begin work on high-value subsystems like the Native C Module and CRC error checking.

In the end, we chose Electron because of our existing familiarity with web-development concepts. By choosing a framework aligned with the team's core competencies, we mitigated technical risk and ensured that the majority of the development effort was spent on high-value engineering challenges, such as real-time data integrity and hardware-software synchronization.

4.3 Proposed Design

4.3.1 Overview

Our project consists of three large groups that interact with each other: the user, the desktop application, and the drone itself. The User can interact with the drone in two different ways: via the desktop application, or directly via the controller.

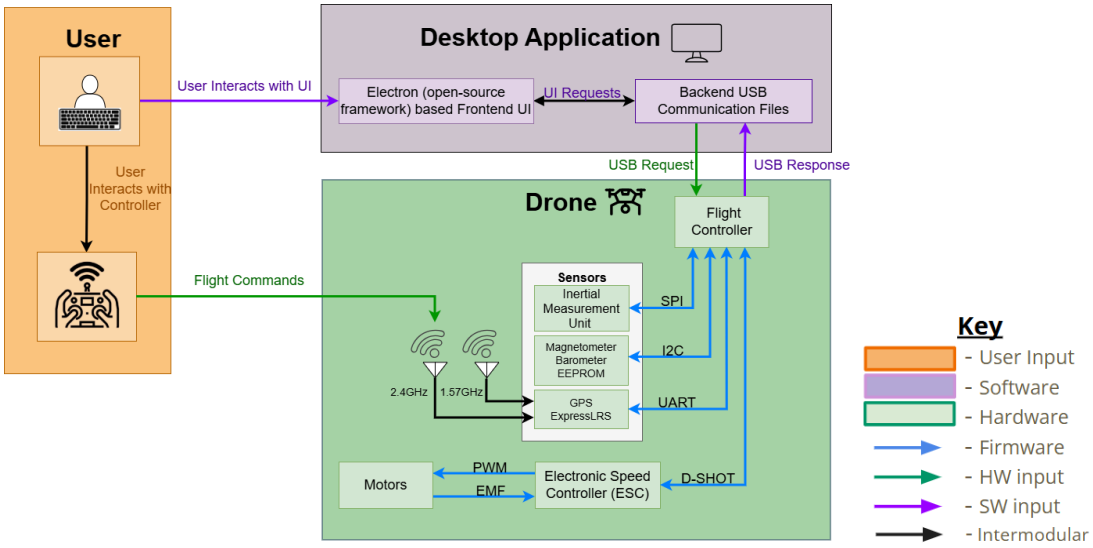


Figure 5. High Level project diagram.

The desktop application acts as an interface between the user and the drone. By interacting with the user interface, the user can view current settings, observe telemetry readings, and update / configure drone settings to their preferences. User interactions trigger frontend requests, which are fed to the desktop application's native C node backend file, which then feeds the request to the flight controller firmware via a COM port through the USB connection. JSON responses from the flight controller firmware are sent back up through this chain to the frontend javascript files, which update the user interface with the data received.

The user can interact with the drone directly via the controller. The controller can give the drone various flight commands. The commands are sent over USB to the firmware on the flight controller microcontroller. The commands are processed and a response is sent back once the command has been executed.

The flight controller takes in commands from the controller and creates DSHOT protocol throttle

requests. Those requests are processed on the electronic speed controller microcontroller and turned into PWM signals to drive the motors. The motor sends back-EMF that allows the reading of RPM and other data. That data is sent back to the flight controller for further processing. The IMU, magnetometer, and barometer are used to help control the drone to achieve steady level flight.

4.3.2 Detailed Design and Visual(s)

Hardware:

The Flight Controller high level is shown below.

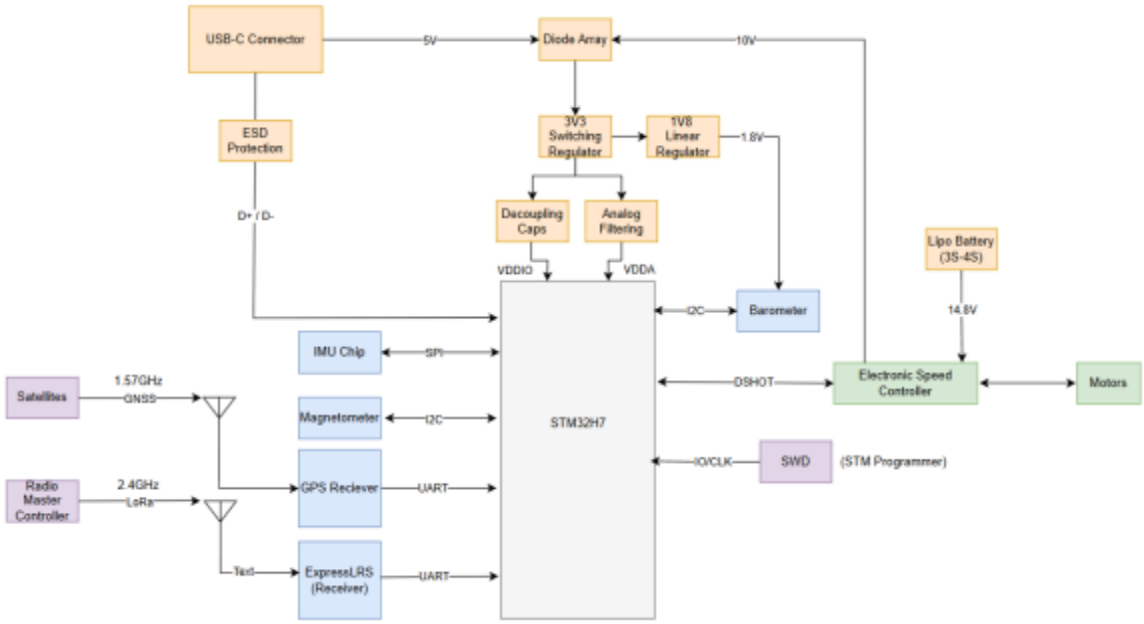


Figure 6. High Level diagram of the flight controller.

Power enters from one of two sources. USB-C delivers 5V during bench work and configuration; the ESC delivers 10V which is regulated down from a 3S-4S LiPo (11.1V-14.8V nominal) during flight. Both feed a dual Schottky diode array that selects whichever rail is higher and blocks reverse current into the inactive source. The OR'd output drives the 3.3V switching regulator, which supplies the MCU's VDDIO through bulk decoupling and VDDA through a separate analog filter network. A 1.8V LDO branches off the 3.3V rail to supply the ENS220 barometer.

The STM32H753 sits at the center. On-board sensors connect locally: the ICM-40609-D IMU over SPI, and the barometer and magnetometer over I2C. USB D+/D- routes from the USB-C connector through a USBLC6-2 ESD array to the H7's OTG_FS peripheral. Four DSHOT lines run to the ESC, which in turn drives the four motors. An SWD header exposes SWDIO/SWCLK for the ST-Link programmer.

Two RF paths terminate off-board and are shown for context only. The 1.575 GHz GPS L1 signal reaches the TESEO-LIV3R through a patch antenna on the GNSS daughter board, which returns position to the H7 over UART. The 2.4 GHz ExpressLRS link terminates on the receiver module and returns control inputs to the H7 over a second UART.

The ESC design is shown in the block diagram below.

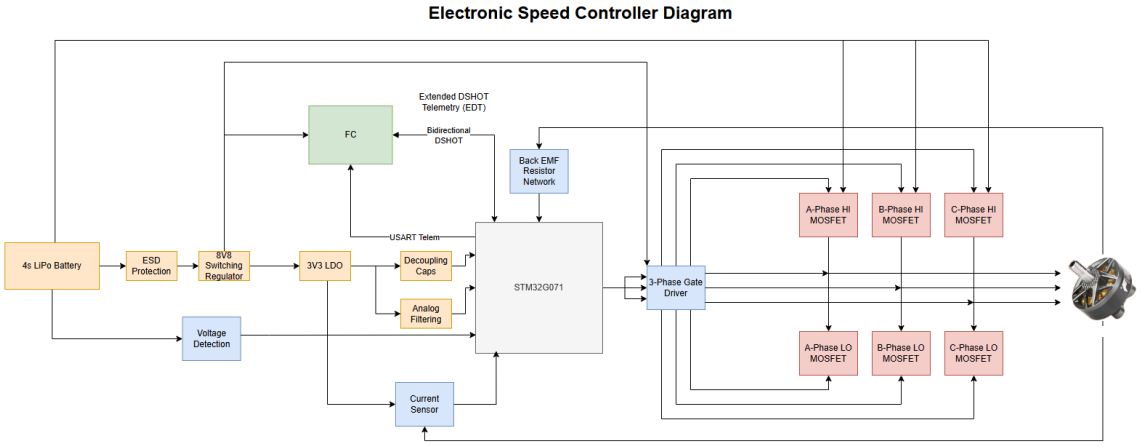


Figure 7. High level diagram of the electronic speed controller

Power is supplied by the 4S LiPo battery. That voltage is then regulated with a 10V switching regulator, which feeds into the FC, Gate Drivers, and the 3.3V LDO. The 3.3V LDO will supply power to the microcontroller. To generate a motor spin, the microcontroller will receive a DSHOT signal from the FC that corresponds to a given throttle value. Based on that throttle value, the microcontroller for that ESC will output three PWM signals, which will be sent to the Gate Driver for amplification to the MOSFETs. The MOSFETs are responsible for switching at the appropriate times to output a pseudo-3-phase sine wave. There is feedback from the motors to the ESC microcontroller via the back-EMF resistor network, letting the MCU know the current position of the rotor, which will be used for more precise motor control.

Firmware:

USB:

The USB functionality is implemented using the TinyUSB driver. This is an open source driver that allows for easy per application customization. The current setup for USB communications is in the form of a master slave interface. The user's desktop application acts as the master with the flight controller being the slave. Commands are only sent from the desktop to the flight controller not visa versa. The flight controller always sends back and acknowledgement response indicating that it received the message and did what it needed to do. Each main customization has an associated command. This allows each command to have a unique response so the desktop application can identify that the correct command is being responded to.

Our transmissions are broken down into 64 byte packets. Each transmission is signified complete after a zero length packet is sent. This allows consecutive packets to be sent to allow larger than 64 byte transmissions. The TinyUSB driver handles this implementation. The first 2 bytes of each packet signify the command or response header. This is a unique hexadecimal code that corresponds to the specific command. For instance, 0x800 is our handshake command and its associated response header is 0x5000. Following the header bytes is a 4 byte data length value. This encapsulates the size of the raw data and a CRC calculation. Inclusion of this packet is to make

parsing quicker to increase responsiveness. Following the length value is up to 56 bytes of raw data. This is any value that may need to be set or sent to the desktop. Lastly, we include two bytes of CRC using the ccitt_16 algorithm. This algorithm is performed on the entire packet minus the last two bytes where the CRC would be placed.

When a transmission has been received a CRC calculation is first performed to ensure we had no data corruption. If a problem is found it immediately sends back a packet with a CRC error header. If the CRC was good we read the command header and verify it is a supported command. If it is found to be an unknown command an unknown command header is set for the response and sent back. Upon successful command processing the firmware executes the required task and sends back the corresponding response header. The following diagram details this logic:

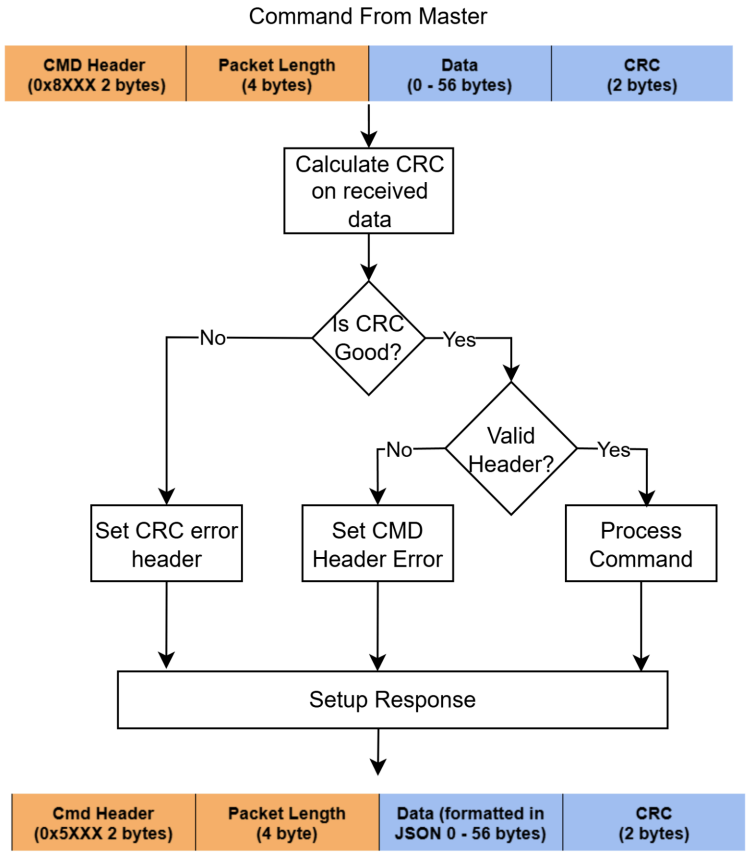


Figure 8. USB protocol flowchart

The logic from above is our main communication with the desktop application through a virtual COM port. Using TinyUSB we have established a second COM port that is used for continuously streaming data. In this application the data being streamed is constant quaternion data that represents the positioning of the drone. This COM port has a similar setup with the master slave interface and its own set of commands. The main purpose is to have the two channels separate so that normal commands can still flow through while data is streaming.

Upon receiving a start streaming command a semaphore is upticked to allow a OS task to begin an infinite loop of streaming. This start command also sets a streaming state variable. Once the stop command is processed the streaming loop is exited to return to a loop that runs on the semaphore downtick. The loop will continuously run in that task until the semaphore is upticked again by a start command. The utilization of the semaphore here helps to keep the CPU overhead down.

Main peripherals:

Inertial Measurement Unit (IMU):

For our design we chose the ICM 40609 IC for our IMU. It provides accelerometer and gyroscope data at 16KHz. This is the primary motion sensor for the flight controller and direct input to the 8KHz PID loop. The steps for initialization can be seen in the diagram below. We check the ChipID to establish connection, then we did a soft reset on the system. We then check the ChipID again, and put both the accelerometer and gyroscope in low noise mode. Then we configure the accelerometer to have 16KHz data rate with a 32g range, and the gyroscope is configured to with a 32KHz data rate and a range of 200 degrees per second.

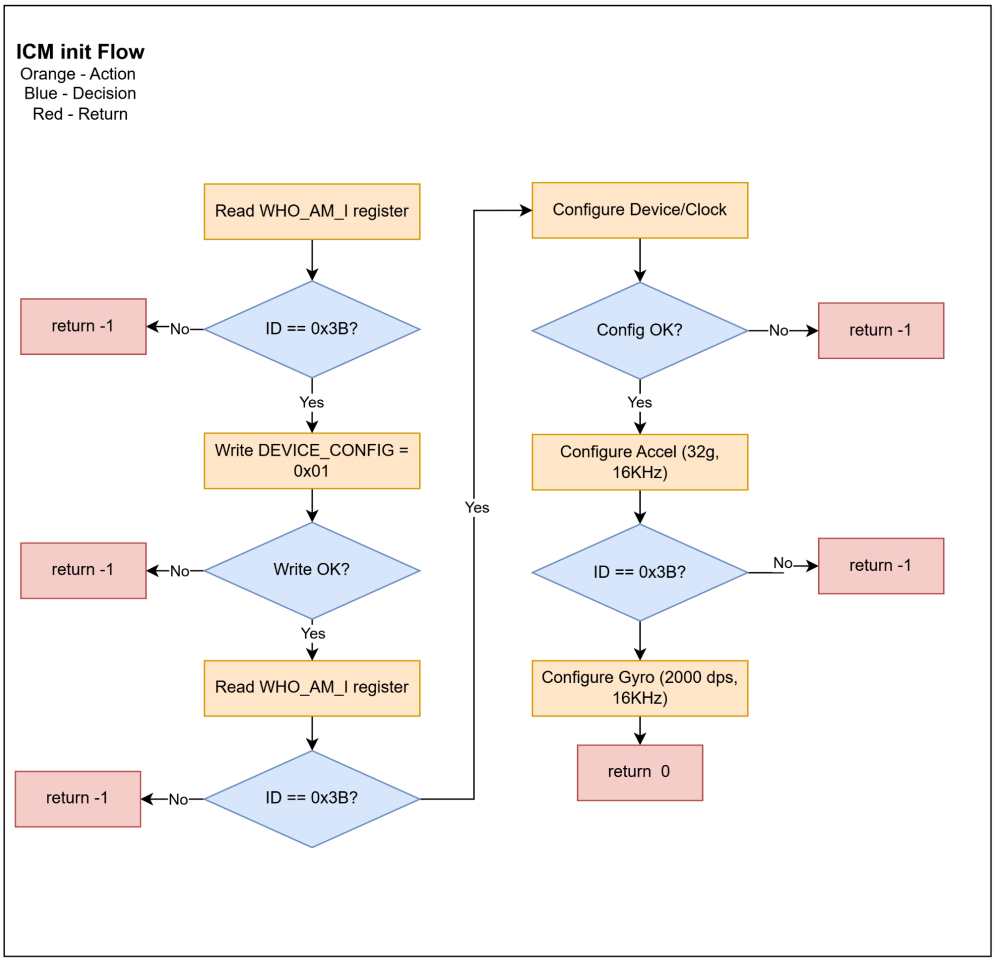


Figure 9. IMU ICM-40609 initialization flowchart.

Magnetometer BNO055:

The BNO055 is a secondary gyroscope initialized after the ICM, although it is a full 9 axis IMU, it has a slower data rate than the ICM and the team is mainly using the BNO055 for the magnetometer values. The sequence is quite longer due to the complexity of the device and the waits between writes and reads were added at the suggestion of the BNO datasheet. The ChipID is read, and written to the system trigger register to initiate a trigger/wake sequence. The ChipID is read again to confirm device stability. The operating mode register is written to put the device into configuration mode. The system trigger register is then written to again. The Sys error register is read in order to check that the init flow is successful. The page id register is written to in order to ensure register page 0 is selected.The power mode register is written to in order to put the device in normal power mode. The operating mode is then written to in order to put the device in magnetometer only mode.

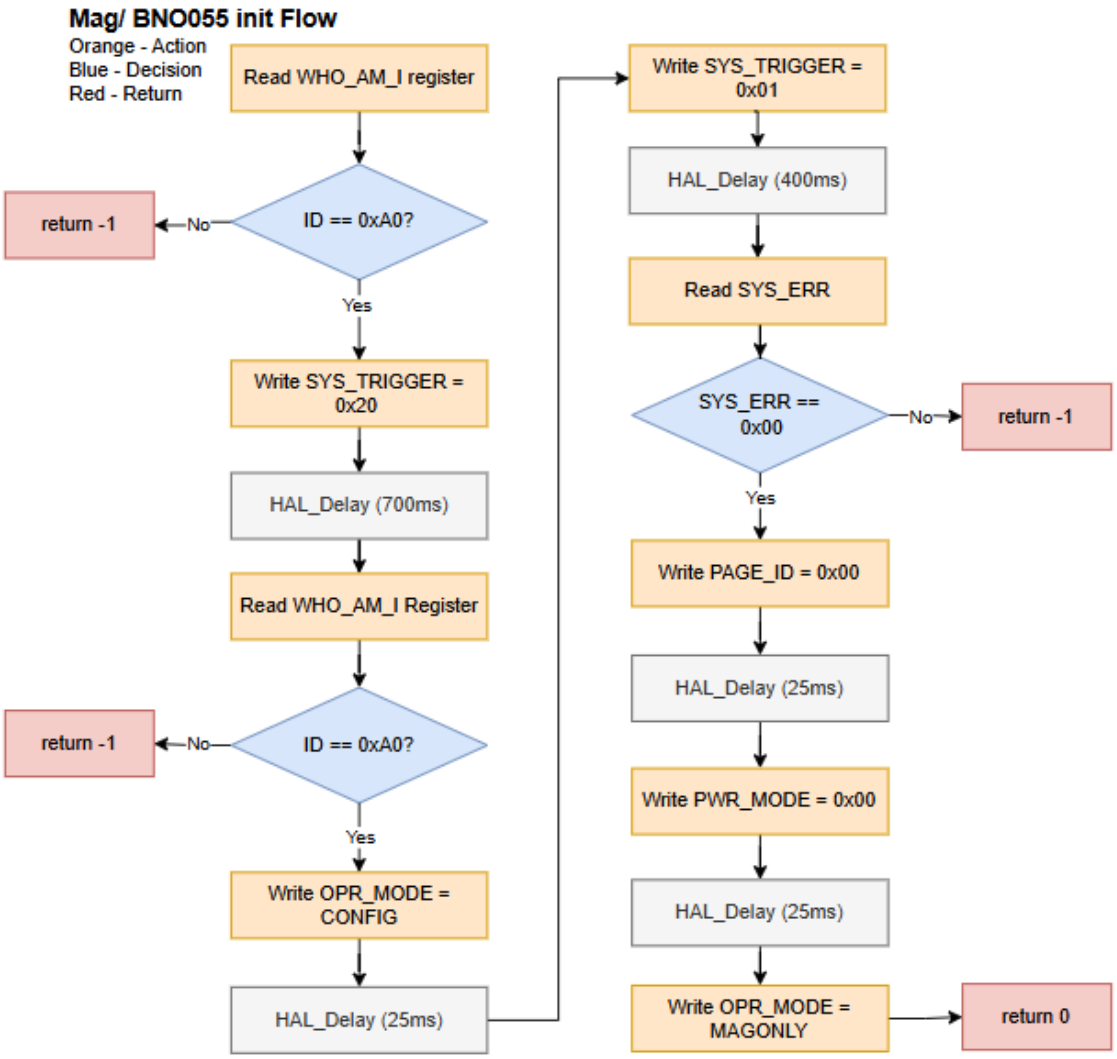


Figure 10. IMU BNO055 Initialization sequence to use the magnetometer only.

Barometer:

For the Barometer we used the ENS220, and the init flow chart can be in the image seen below. The ChipID is read, then the mode configuration register is used to place the device in standby mode. We then perform a soft reset. The measure configuration register is read to retrieve the current measurement state. It is masked and modified to configure the measurement rate and mode. The mode configuration is written in order to place the continuous measurement mode.

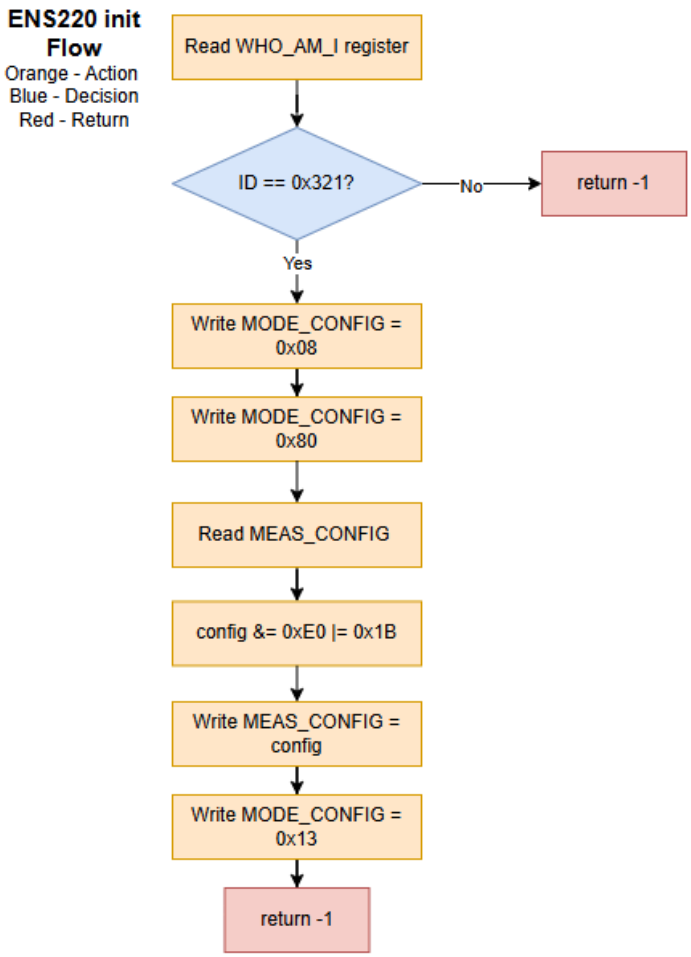


Figure 11. Barometer ENS220 initialization flowchart.

Software:

Desktop Application:

The Desktop Application is the primary interface between the user and the drone. This application uses both command-based and streaming-based data communication and monitoring for a stationary drone. The system is designed with a layered architecture style that separates the user interaction, communication, and flight controller interaction into several distinct components. These components are:

- Frontend (User Interface) Application Layer:
    - Built using Electron, the frontend provides an interactive way for users to send commands, adjust parameters (such as throttle values), and visualize incoming data.
    - It is responsible for capturing user input and displaying results but does not interact directly with the drone. It instead interacts with the IPC mechanisms in the IPC Layer.
- Inter-Process Communication (IPC) Layer:
    - The IPC Layer acts as a bridge between the frontend and backend processes. It enables the frontend to request backend operations and receiver results while keeping processes separated.
    - The IPC mechanisms support both request-response communication and event-driven communication for continuous and real-time data streaming.
- Backend (Native Communication) Layer:
    - The backend handles all communication with the flight controller. It consists of a native C module integrated into the Electron application that manages all serial communication over USB.
    - The backend is responsible for constructing and transmitting communication packets, receiving and processing responses from the flight controller, validating incoming data and handling continuous streaming data from the device.

These components are interconnected so that user actions in the frontend trigger IPC messages which are then sent to the backend where the backend handles communication with the flight controller. Responses and streaming data follow the reverse of this path which allows the frontend to update the UI in real-time.

To support both responsiveness and performance, the system is designed with concurrency and ease-of-use in mind. Backend operations and IPC messaging occur independently of the frontend rendering thread, and streaming data is processed on a separate execution thread than standard communication is handled on. This ensures that continuous data updates do not interfere with user interactions or commands. This architecture provides a modular and scalable design that cleanly separates different subsystems so that they can be developed, tested, and maintained independently. An illustration of this can also be seen in the **figure below.**

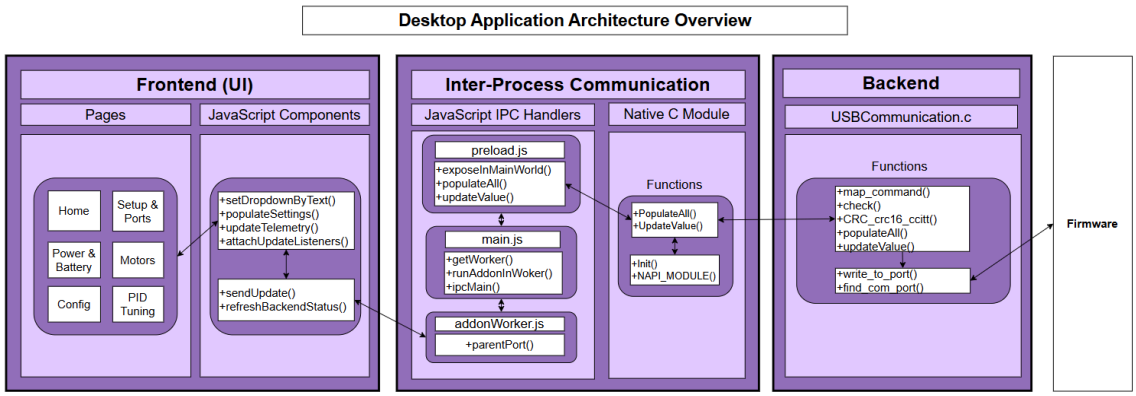


Figure 12. Desktop application architecture overview.

Frontend:

The frontend of OSCAR Ground Station, within the Application Layer, consists of the visual interface that the user interacts with. The user interface is designed using HTML and CSS. Each of the elements (buttons, dropdowns, inputs, etc.) that the user can interact with is designated a unique request description (i.e. “BatteryCapacity”) that is referred to in its page’s associated javaScript file to tell the backend what request is being made. The backend will translate this description string into a request packet with a unique header and data. This translation is described in more detail in the Backend section.

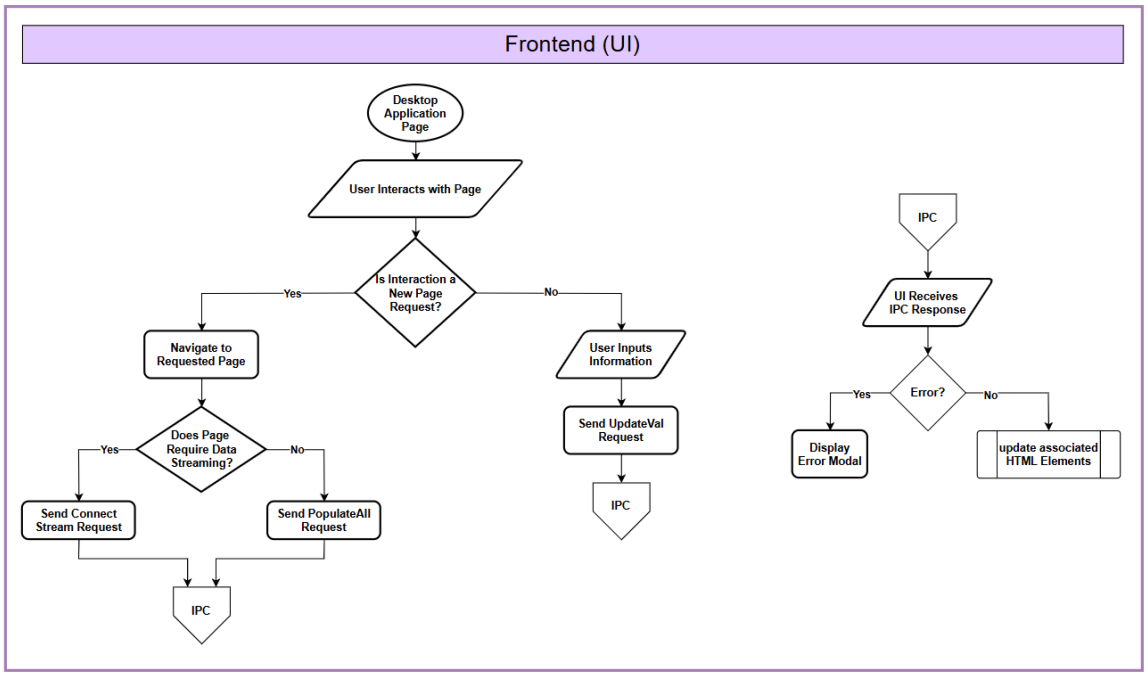


Figure 13. Frontend UI flowchart.

As soon as a new page loads, the frontend javaScript will attempt to refresh the backend API status. This is done by contacting the IPC files, which is described in more detail in the IPC section. If the backend API is not available, an error will be displayed on the console. If the backend API is available, the frontend will then begin the communication process. If the page requires data streaming as well as standard request–response interactions, the frontend will send a request to connect to the data stream. If the page does not require data streaming, it will request for the entire page to be populated with the current relevant data on the flight controller.

Once a page is loaded and either has streaming established or the page data populated, users can interact with the elements to edit drone settings. For example, a user could press the “Calibrate Gyroscope” button on the Setup & Ports page. This would send a single update value request to the backend, which would forward the request to the flight controller firmware to invoke the gyroscope calibration sequence.

Once a request is sent to the IPC Layer, the frontend will wait for a response. Responses come in the form of JSON objects from the flight controller firmware. The responses are first received by the

backend, where they are evaluated for errors using the CRC checks. If the backend detects any errors, it forwards an error message through the IPC to the frontend, where an error modal will be displayed with an associated error message to alert the user. If there is no error, the frontend will parse the JSON response for the relevant data, and display the data to the screen.

The frontend data flow can be observed using the flowchart **above**.

Inter-Process Communication:

Inter-Process Communication (IPC) is used to manage data exchange between the frontend and backend processes within the Electron desktop application. Because Electron separates the user interface from lower-level system operations for security and stability, the frontend cannot directly access native modules. IPC provides a controlled messaging layer that allows the frontend to request backend functionality and receive results.

These IPC interactions occur independently of the frontend rendering thread. This ensures that user interface updates remain responsive and are not blocked by backend operations such as serial communication or data processing.

For standard operations, IPC follows a request–response pattern. When a user interacts with the interface, the frontend sends a message to the backend containing:

- A command identifier specifying the requested operation
- Any required data or parameters associated with the request

The backend listens for incoming messages, processes each request, and returns a response. This response is then forwarded to the frontend, where the UI is updated accordingly.

Streaming data is handled differently from standard request–response interactions. Instead of the frontend requesting each update, the backend continuously sends data as it becomes available. Once streaming is enabled, the backend listens for incoming data on the secondary COM port and forwards parsed data to the frontend using an event-based messaging approach.

Streaming and standard command handling operate on separate execution threads. This separation ensures that continuous data streaming does not block or delay discrete command processing, allowing both operations to run concurrently without impacting application performance or responsiveness. Please **view the image below** to see a visualization of the flow of data within the IPC mechanisms.

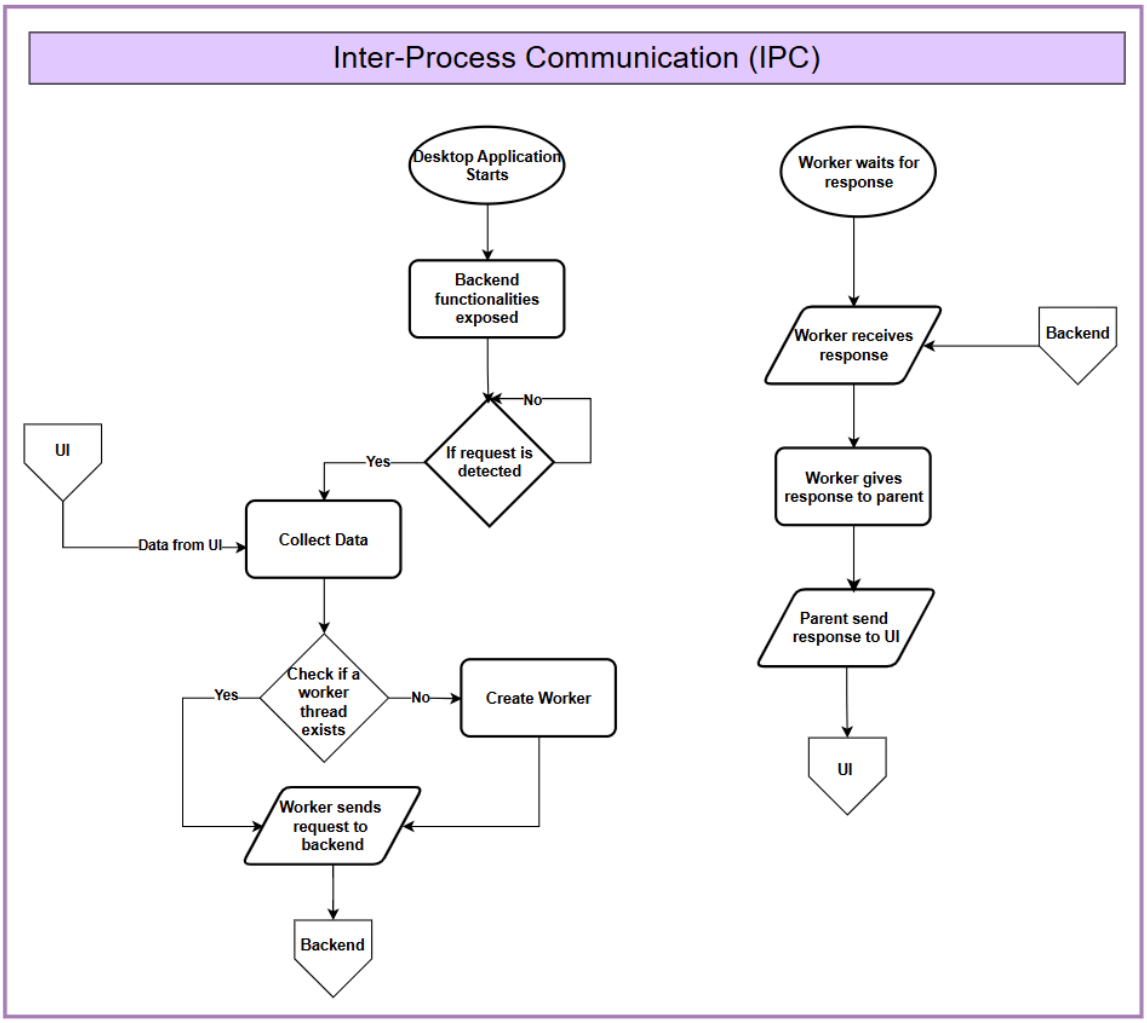


Figure 14. Inter-Process Communication flowchart.

Backend:

All communication between the Flight Controller and the Desktop Application is handled by the backend subsystem. Backend operations are initiated by the frontend through Electron IPC handlers, which act as the interface between the UI layer and native functionality.

The backend consists of a native C module that uses the libserialport library to communicate with the device over a USB COM port. Because Electron applications run in a JavaScript environment, this C module is precompiled and integrated into the project as a native addon. To enable communication between JavaScript and the native module, wrapper functions are implemented using the Node-API (N-API), providing a stable bridge between the frontend and the C backend.

When the frontend issues a request, it invokes a corresponding IPC handler, which calls into the native module through the Node-API wrapper. The C module then determines the request type and performs the following steps:

1) Converts the given string command into is hexadecimal header value

2) Processes the incoming payload from the frontend
3) Constructs a communication packet that includes:
    a) Header
    b) Packet Length
    c) Payload Data
    d) CRC (Cyclic Redundancy Check)
4) Transmits the packet to the flight controller via the serial port

Upon receiving a response from the flight controller, the backend then:
1) Validates the packet using CRC checks
2) Parses the returned data
3) Converts the data into JavaScript-compatible format
4) Returns the result to the frontend through the IPC response

Most backend operations follow this request–response pattern, with the exception of data streaming. Streaming data is handled differently from standard request–response interactions. Instead of terminating communication after a single response, the system establishes a continuous data stream using a secondary COM port.

When streaming is initiated:
- A second serial COM port is opened
- A listener loop continuously reads incoming packets
- Parsed data is forwarded to the frontend using a callback mechanism

Streaming control (start/stop) is managed through commands sent over the primary COM port. When a stop command is issued:
- The backend signals the flight controller to stop streaming
- The listener loop is terminated

The streaming listener runs on a separate execution thread from standard request–response operations. This ensures that continuous data streaming does not block or delay command handling. To prevent race conditions when accessing shared resources, synchronization mechanisms such as mutex locks are used. These ensure safe and consistent access to shared data between threads.

The streaming listener operates in a non-blocking manner, allowing continuous data reception without interfering with standard request–response communication. This design allows streaming to occur simultaneously with regular update commands and requests. This process can be seen visually in the flow chart **provided below**.

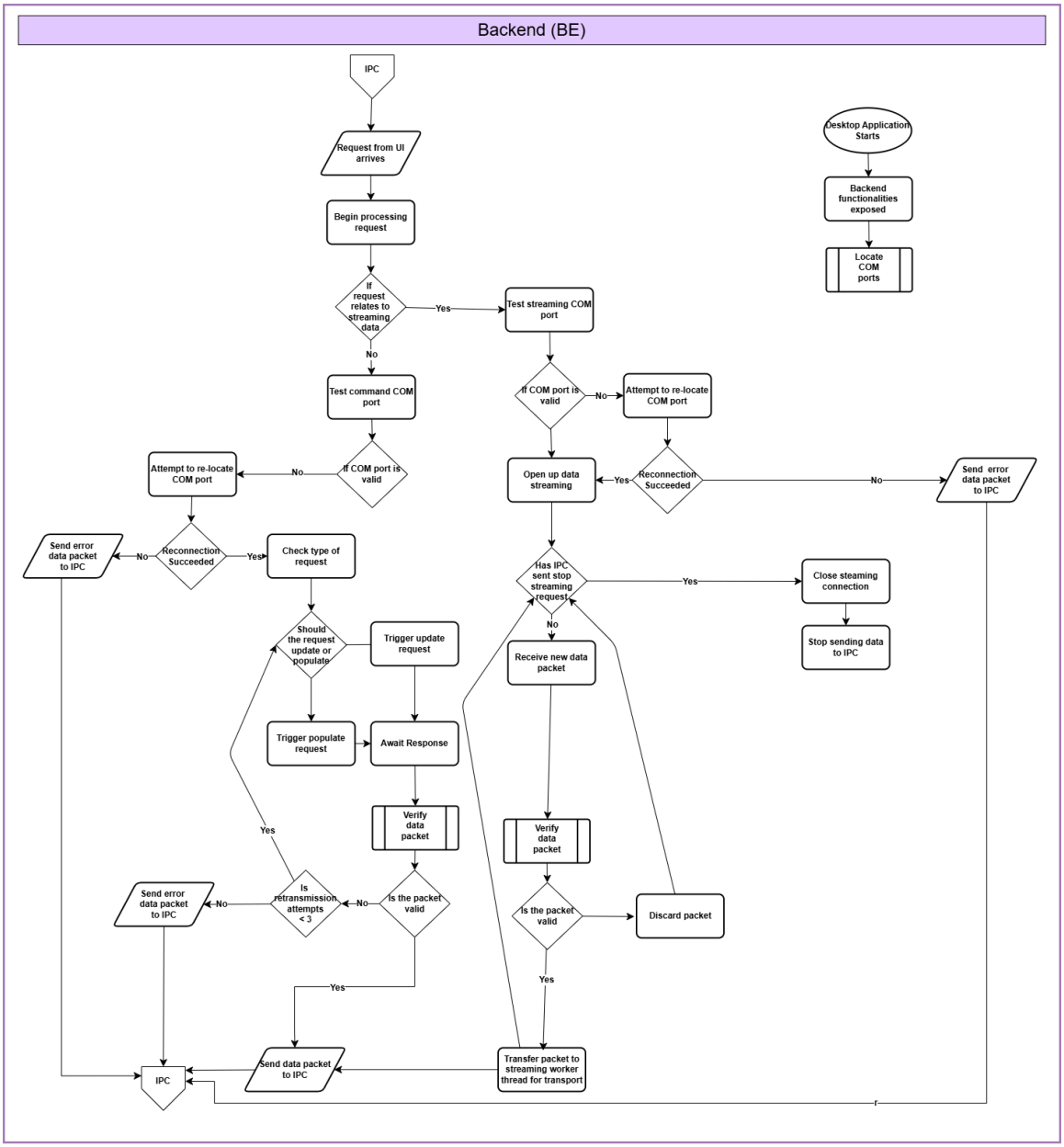


Figure 15. Backend flowchart.

4.3.3 Functionality

There are three main aspects to the OSCAR project: Open-source documentation (OSCAR Docs), the accompanying desktop application for configuration (OSCAR Ground Station), and the drone itself.

1) A user (presumably a student, researcher, or drone hobbyist) would refer to the OSCAR Docs website for detailed instructions on drone customization, configuration, and operation; from the hardware components of the drone itself, to the firmware on the flight controller, to the software for the application.
2) A user would then download the current version of the OSCAR Ground Station application, available as an executable. They would use this to customize/view the settings of their drone; such as component calibration, component configuration, telemetry readings, motor testing, PID tuning, and receiver/GPS configuration. This configuration is done via a UART protocol over USB between the drone flight controller and the user's PC with the downloaded executable.
3) Once the user has customized the drone to their preferences, they can disconnect it from their PC and fly it with the RadioMaster controller.

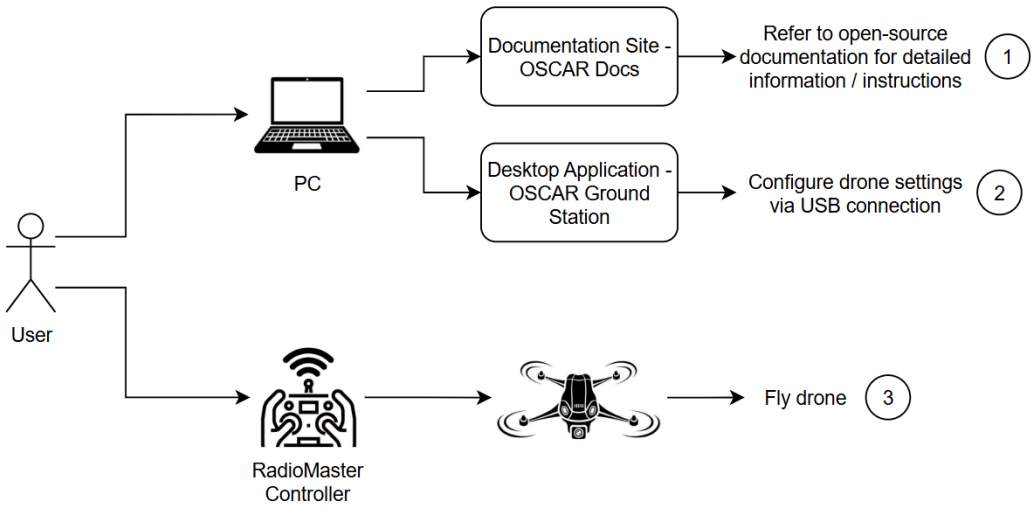


Figure 16. OSCAR use-case example flowchart.

4.3.4 Areas of Concern and Development

Hardware:

From our current understanding, our design will satisfy all our requirements. This can't be confirmed until we have finished our tests on the hardware. The control system has not been tested with our specific design and there is always a possibility for an oversight during design that isn't caught until testing. Concerns related to hardware specifically are the power dissipation abilities of the ESC board and whether overheating is a concern. The math for calculating this was done using estimates and the design was made to have a large margin of error to allow for mistakes in the calculation, though this is still an estimation.

Firmware:

The current design for the firmware is on track to meet the needs of the wide audience range. Our initial designs are providing proper interfacing with external software and are making good headway for inter-controller communication. Eventually the current design will allow for the full fledged customization of different control parameters. Currently we have the bones in place to allow this. We just need the supporting hardware to fully integrate and test our implementations.

The main concern with the current design is it is mostly proof of concept through development boards rather than the fully custom hardware. There are different obstacles that become apparent when working with the real hardware versus the well proven development kits. Moreover, there is concern of integration with the software team. There are many different parameters that will need to be supported and we will need to make sure none of them fall through the cracks. If some customizations are missing it could lead to a more harmful user experience.

Some immediate plans include working with the custom hardware as soon as its available. Working with integration as soon as possible will help to mitigate potential time constraint issues. For the concern with software team customizations, we plan to communicate early and often to ensure nothing gets left out.

Software:

The current design of the Desktop Application from the Software team is on track to satisfy the requirements and user needs. The design and architecture of the project has proven to be sound, communication with the Flight Controller is functional, all UI pages have been built, and some have been fully integrated with USB communication. The only real gap that remains is ensuring all pages are populated with USB data.

The primary concerns going forward are integration with the Hardware and Firmware teams, usability, and testing coverage. Since the software relies on the Flight Controller for data population, significant changes on the firmware side could impact progress made in the application. Usability is also a priority given that one of the main goals of our Desktop Application is to be accessible to newcomers. Finally, more thorough testing will be needed to discover bugs and edge cases that have not yet been covered.

Immediate plans include staying in communication with the Firmware team on USB protocols, setup, and commands, as well as beginning to test the application for bugs and discovering edge

cases. The team would also welcome feedback on the UI design to gauge how intuitive and accessible the interface really is.

4.4 Technology Considerations

Hardware:

For our hardware design we will be using printed circuit boards with both surface mount and through hole components. Printed circuit boards are an incredibly useful technology for component density, reliability, and robustness. The main weaknesses are the increased expertise when designing as well as manufacturing. Another weak point is the amount of time required to order and receive the board which makes prototyping much slower. Alternatives include using breadboards for development and proto boards for long term but larger solutions. These prototyping boards lack the robustness and small profile of PCBs which is critical for our drone design. These tradeoffs are not worth it in our case.

Firmware:

Table 7. Embedded technologies and their roles within the flight controller system architecture.

Technology	Role
FreeRTOS	Operating system to manage timing
TinyUSB	USB driver to handle usb communications and establish a dual COM port
DShot	Communication protocol to send motor commands at high speed
I2C	Communication protocol used with low speed peripherals such as the magnetometer, barometer, and EEPROM.
SPI	Communication protocol used for high speed data transfer between the microcontroller and peripherals such as the IMU.
ExpressLRS	Communication protocol used for data transfer between the controller and the radio master on the flight controller.

FreeRTOS

- *Strengths*: It is an open-source, real-time operating system targeted towards microcontrollers and microprocessors.
- *Weaknesses*: It is difficult to debug. When there is an issue, it often silently errors out and loops forever making it difficult to find the root cause of the issue.
- *Tradeoff*: We traded the ability to easily diagnose issues for the ability to schedule and prioritize tasks as well as transparency in terms of implementation.

TinyUSB

- *Strengths*: It is open-source and designed for microcontrollers. It is able to accommodate multiple COM ports over the same USB connection without lots of code changes.
- *Weaknesses*: It is not an innate library to STM32 and we had to spend time integrating it when there was a built-in driver for USB communication. However, the built-in USB driver could not easily handle multiple COM ports.
- *Tradeoff*: We traded the time spent adding and debugging TinyUSB to work with our current firmware for the ability to continuously stream data on a second COM port.

DShot

- *Strengths*: It is one of the fastest communication protocols for communication to ESCs. As a digital protocol, it is significantly less affected by noise and because of that, it does not need calibration.
- *Weaknesses*: DShot is not compatible with older ESCs that require analog signals and they have a fixed frame length, so there is no efficiency gain at lower throttle settings.
- *Tradeoff*: We traded a small amount of speed and packet efficiency for reliability and noise resistance.

I2C

- *Strengths*: Most of the peripherals we are using all communicate through I2C and SPI, so we are able to reuse code. It also allows multiple device communication on a single bus and operates off the real time clock.
- *Weaknesses*: It is a low speed communication protocol.
- *Tradeoffs*: We are trading speed for simplicity of hardware design and broad usage across the microcontroller.

SPI

- *Strengths*: Most of the peripherals we are using all communicate through I2C and SPI, so we are able to reuse code. It also is a high speed communication protocol.
- *Weaknesses*: It is a more complex hardware design and requires more pins than I2C.
- *Tradeoff*: We are trading simplicity of hardware design for speed and broad usage across the microcontroller.

ExpressLRS

- *Strengths*: Open-source, high refresh rate radio control communication protocol with longer distance communication capabilities.
- *Weaknesses*: There is a steep learning curve to get everything set up.
- *Tradeoff*: We traded time to get the radio master setup and communication with the controller for long distance, fast, reliable communication from controller to drone.

Alternative for DShot: MultiShot

- *Why considered*: It is the fastest communication protocol used for ESC communication.
- *Why rejected*: It is an analog protocol so it is much more susceptible to noise and requires a high-performance ESC to make use of its speed.

Alternative for TinyUSB: STM32 USB driver

- *Why considered*: It was already written and native to the IDE we were using. It didn't take very much time to set up.
- *Why rejected*: It required a lot of manipulation to be able to handle multiple COM ports.

Software:

The software tech stack used for the Software Team is described in the table below:

Table 8. Software architecture and technology stack for the UAV ground control system.

Category	Technology	Role
Application Framework	Electron	Provides the cross-platform environment and multi-process architecture.
Runtime Environment	Node.js	Handles system operations and enables backend logic within Electron.
3D Rendering Engine	Three.js (WebGL)	Real-time visualization of drone orientation and telemetry.
UI Technologies	JavaScript, HTML, CSS	Implements the user interface and interaction layer.
Data Compression	fflate	Efficient compression and decompression of transmitted data including 3D model assets.
Native Bridge	Node-API	Connects JavaScript layer to native C code using napi_value interface.
Serial Communication	libserialport	Enables USB/UART communication with the flight controller and configuration of COM port settings.
Data Integrity	CRC-16/CCITT	Mathematical validation of binary packets received from the flight controller.

Hardware Bridge	Native C Module	High-performance, low-level hardware interaction.
Build & Distribution	Electron Forge and Squirrel	Packaging, building, updating, and distributing the application for a windows operating system.

Various strengths, weaknesses, and trade-offs for our chosen tech stack are described below:

Electron

- *Strength*: Rapid UI development and a multi-process model that keeps the 3D render loop separate from the data-parsing logic.
- *Weakness*: High memory overhead (RAM) and large binary sizes due to the bundled Chromium engine.
- *Trade-off*: We traded system resource efficiency for development velocity and UI flexibility. Since drone pilots typically use modern laptops, the ~200MB RAM footprint was deemed an acceptable cost for a superior, 60FPS graphical interface.

Native C Module (N-API)

- *Strength*: Near-metal execution speed for processing high-frequency serial data and calculating CRC checksums.
- *Weakness*: Increases build complexity; the module must be recompiled for every target OS (Windows, Mac, Linux).
- *Trade-off*: We traded code simplicity for timing determinism. Handling UART interrupts and bit-level CRC in pure JavaScript would risk latency spikes during garbage collection; C ensures consistent data processing.

Possible alternatives to the Electron framework with Native C Module were considered:

Alternative A: Python with PyQt/PySide

- *Why considered*: Python has excellent serial libraries (pyserial) and is popular in the drone community.
- *Why rejected*: Implementing a high-performance 3D viewport (like Three.js) in Python is significantly more difficult and less performant than WebGL-based solutions.

Alternative B: Tauri (Rust-based)

- *Why considered*: Tauri creates much smaller binaries and uses less RAM than Electron.
- *Why rejected*: The team lacked proficiency in **Rust**. Given the project deadline, the time required to learn Rust's ownership model to build the C-interop layer posed a high risk to the project schedule.

Alternative C: Pure C++ with Qt

- *Why considered*: Maximum performance and industry-standard for professional GCS (Ground Control Stations).
- *Why rejected*: The overhead of manual memory management and the complexity of building a modern, CSS-styled UI would have slowed down feature implementation compared to the Electron ecosystem.

4.5 Design Analysis

Hardware:

The first revisions of all necessary boards have been ordered and are currently being assembled. The flight controller board has worked exactly as expected in all tests performed thus far. The ESC board has struggled to work because of assembly difficulties. The issues were completely unknown to us before attempting to assemble the first board. This was an assembly issue and not a design feasibility issue. The GNSS board has been assembled, communication to both magnetometer and GNSS has been confirmed with the firmware team, testing of GNSS receiver capabilities has not been tested.

Firmware:

- Communication protocols
    - I2C
    - SPI
    - DShot
    - USB
    - UART
    - ExpressLRS
- OS
    - FreeRTOS
- Peripherals
    - IMU
    - Magnotometer
    - Barometer
- Communication to Software functionalities
    - Continuous streaming over USB
    - Motor Page configuration
    - Power and Battery Page configuration
    - Configuration Page configuration

The proposed design has experienced a lot of change throughout the semester and what is listed in 4.3 is the most updated plan. The peripherals being used have changed often as hardware was designing the boards, which has caused a lot of code development and then throwing out that code to start development for a different peripheral. Additionally, the STM32 USB driver we started initial development on and got working was not easily able to handle multiple COM ports, causing us to pivot to TinyUSB and try to integrate that beneath all the code used to communicate from the Software side to the Firmware controlling various peripherals. Merge conflicts from the ioc file

which is generated by the IDE we are using has caused several silent failures which were difficult to diagnose. These setbacks have caused our scope to tighten in regards to progress expected by the end of semester one.

By the end of this semester we would like to be able to configure and control the motors from the desktop application as well as have the motors respond to the controller. We would also like to have quaternion data able to continuously stream to the desktop application and get EEPROM firmware done. Next semester, we would like to implement the GPS firmware, bidirection DShot, telemetry from the ESC to the flight controller, and roll, pitch, and yaw throttle control.

Software:

So far everything that the Software team has built, implemented and tested includes the following:

- Several UI Pages (only UI has been developed)
    - Home
    - PID Tuning
    - Receiver
    - GPS
- Several UI Pages that populate data based on USB commands
    - Setup & Ports
    - Configuration
    - Power & Battery
    - Motors
- Communication with firmware functionalities
    - COM port connection and verification via handshake commands
    - Update value requests
    - Fetch page data requests
    - Continuous Data Streaming
- Executable file functionalities
    - An executable that installs the application on a windows device
- Open source repository
    - Repository layout and setup
    - Documentation website template - hosted by GitHub pages

The proposed design has worked mostly as expected. The design and architecture decisions made in section 4.3 proved to work very well for the project. Each UI page has been successfully built and integrated, and communication with the Flight Controller has been functioning as intended. This has already allowed approximately half of the implemented UI pages to populate and update data directly from the Flight Controller.

The one area that did not go entirely smoothly was the executable installer. Issues were encountered during the packaging and installation process on Windows devices, requiring adjustments to the build configurations. However, these issues have since been resolved without any design changes being necessary, and overall this was a minor setback in the broader scope of the project.

Looking ahead, the Software team's plans center around three main areas. The primary focus will be implementing USB communication data population and updating across the remaining UI pages.

Secondly, the existing communication systems will be improved by adding more robust error handling, additional commands, and better edge case detection. Finally, a dedicated phase of testing will be conducted, including efforts to gather feedback from individuals outside the group on the UI design. This external input will help with bug fixes and allow the team to address any UI issues before the project is finalized.

5 Testing

5.1 Unit Testing

Hardware Team:

Unit testing follows the same general process across all three boards (flight controller, ESC, GNSS), with board-specific considerations noted below.

Flight Controller:

> After assembly, continuity tests are performed with a digital multimeter (DMM) to verify no short circuits exist on the power rails. Solder jumpers built into the design allow each voltage regulator to be tested individually, isolated from downstream loads, to confirm correct functionality before powering the rest of the board. Once each regulator is validated, the jumpers are soldered closed. The MCU is then verified over SWD using ST-Link to confirm programmability and communication. Test points placed along the board allow communication protocols (SPI, I²C, UART) to be probed with an oscilloscope to confirm signal integrity and correct operation of peripherals.

GNSS :

> Following the same procedure, after assembly continuity check, UART and I2C lines that validated communication were occurring for both the magnetometer and GNSS. The performance of the patch antenna has not thoroughly been completed, we were able to receive data while roaming outside, but no through validation tests have been performed.

ESC Board :

> The ESC is more involved. The continuity check and isolated regulator validation through solder jumpers are performed the same as the flight controller, but verification of the gate-driver and power stage requires additional steps before a motor is ever connected. Verification of output PWMs on each motor output is performed to validate that the microcontroller is outputting a PWM, the gate driver is amplifying it, and the MOSFETs are properly switching. Testing of the Back EMF circuitry is planned next for validation, as well as thermal performance/current handling.

Tools: bench DMM, oscilloscope, current-limited DC power supply, ST-Link, camera microscope

Firmware:

USB communications to Software:

> Our primary tools for testing is the H7 Nucleo board, our flight controller, and the HTerm application. HTerm is a serial port watching program that allows us to see the full data packets being sent. Using the H7 nucleo board or the flight controller board, we flash the newly written firmware on debug mode. Adding breakpoints where applicable, we use HTerm to connect to the COM ports, and send the headers and potential data to test that the appropriate switch case is hit and changes the data as expected. Then, the corresponding response is sent with the appropriate data and confirming that response that is received in HTerm.

Radio Master to Flight Controller communication:

Using the H7 nucleo board or the flight controller, connect the radio receiver to the pins specified in the pinout document, flash the firmware to the board in debug mode. Then we check the ISR tabs in debug mode and move the controller to confirm the register value are changing as expected.

DShot communication from the Flight Controller

Using the H7 nucleo board or the flight controller, flash the firmware to the board in debug mode and connect an oscilloscope probe to each PWM pin specified in the pinout document. We then write a throttle value to the pin, and confirm the PWM wave has a frequency of 600 kHz, and that the time high matches the 1 and 0 specified in the DShot document. We then extract the binary from the PWM output to check the throttle value sent plus the CRC.

Software:

USB communications to Firmware:

Validate USB communication between the desktop application and firmware using a flight controller rev board or H7 Nucleo board. The desktop application connects to the appropriate COM port and exchanges data with the firmware in real time. Testing involves verifying that the application can successfully establish a connection, send commands, and receive responses without errors. Live data logging is used to confirm that messages are transmitted and received correctly, with no noticeable latency or data loss. Additionally, streamed data is monitored to ensure it is processed continuously and that the user interface updates accurately and in real time.

UI Functionality and Usability:

Testing is performed through manual walkthroughs of each UI element to verify correct behavior and visual feedback. Each interactive component is worked with individually to confirm it responds as expected. We verify that buttons trigger the correct actions, dropdowns populate with appropriate options, status indicators update in response to system state changes, numerical fields increment and decrement correctly, and error conditions are surfaced clearly to the user. Any UI that communicates with the firmware is cross-checked against live data logs to confirm the output display matches the transmitted and received data.

Drone Configurations:

The desktop application provides several configuration and monitoring options for a single drone. The user can edit PID tuning values and motor/ESC settings, with changes transmitted to the flight controller in real time and confirmed via live data logging to verify the correct values are received and applied as expected. Additionally, the application provides a gyroscopic view of the drone to visualize orientation, sensor calibration tools for applicable onboard sensors, and a power and battery information display to monitor the drone's electrical status during operation. These will also be tested by connecting to a flight controller to verify all outputs are expected. Finally, feedback will be gathered from users outside of the development group to evaluate the usefulness of the available configurations and identify any additional features or adjustments that would better suit the needs of a broader user base.

Open-Source Documentation Usability:

All project documentation is hosted publicly on a GitHub pages website. Testing focuses on two primary goals: ensuring the content is easy to read and follow, and ensuring that users can navigate the site easily and intuitively. To evaluate this, individuals outside the development group will be asked to read through different sections of the documentation and provide feedback on whether the material is clear, well-organized, and easy-to-navigate.

5.2 Interface Testing

Hardware Team:

Interface testing validates the points where two or more boards interact under realistic load.

Power Distribution (ESC → FC → GNSS):

The ESC board supplies regulated power to the FC, which in turn supplies the GNSS board. Regulators are load tested while feeding their actual downstream boards. Monitoring of the output voltage ripple on an oscilloscope allows us to confirm they hold within tolerance under combined current draw.

DSHOT (FC ↔ ESC):

The DSHOT protocol between the flight controller and ESC is verified by capturing the signal with an oscilloscope, confirming bit timing, frame structure, and that commanded throttle values produce the expected ESC response.

Firmware:

External Controller to Flight Controller Communication via ExpressLRS:

The interface between Flight Controller and external controller was tested through STMCubeIDE. The ExpressLRS module was connected via UART to the flight controller, and we read values out of the registers directly in order to debug. There were values we were expecting, so after reading the values we were able to debug from there.

Software:

Desktop Application to Flight Controller Communication:

The interface between the desktop application and the flight controller is tested by connecting a flight controller revision board or H7 Nucleo board to a computer running the desktop application via USB. Testing verifies that the application successfully establishes a connection to the correct COM port, transmits commands to the firmware, and receives responses in real time. The firmware side is monitored using CubeIDE debug mode to confirm that requests are being received and processed correctly, while the desktop application's live data log is used to confirm that responses are accurately displayed on the software side. Both sides are checked simultaneously to ensure data is flowing correctly across the interface in both directions with no errors or data loss.

Desktop Application:

The internal interface between the UI components and the underlying application logic is tested through manual walkthroughs of each UI element. Testing verifies that interactive components such as buttons, dropdowns, numerical inputs, and status displays correctly communicate with the application's backend and update as expected in response to user actions and incoming data. Where UI components interact with the firmware, their outputs are cross-checked against live data logs to confirm that what is displayed matches what is

transmitted and received. This ensures that the UI layer and the application logic are correctly integrated and behaving consistently.

Open-Source Documentation:

The documentation hosted on the GitHub Pages website is tested as an interface between the project and its end users. Testing evaluates both the navigability of the site and the clarity of the content by having individuals outside the development group read through select sections and provide feedback. They will assess whether the site structure allows them to find information easily and whether the documentation itself is clear and organized well enough for a new user to understand the setup and operation of the system. Feedback is collected and used to identify any gaps, unclear steps, or navigation issues that need to be addressed.

5.3 Integration Testing

Desktop Applications to Flight Controller Communication

To test this, Software creates an .exe file and Firmware flashes the current firmware to either the H7 nucleo board or one of the two Flight Controller revision, then the flight controller is plugged into the computer with an open instance of the .exe file and test that Software is receiving a response from the board, that the data they receive makes sense in the context of the request, and on the Firmware receives the requests from Software and the data changes on both sides are occurring as expected.

Radio Master to Flight Controller Communication

Use the H7 nucleo board or the flight controller. If using the nucleo board, connect the radio receiver to the pins specified in the pinout document, flash the firmware to the board, run the project on CubeIDE in debug mode, check the ISR tabs in debug mode and move the controller to confirm the register values are changing as expected. Additionally, check that with a change in controller throttle values, a corresponding DShot PWM is output.

Flight Controller to Electronic Speed Controller Communication

Use the H7 nucleo board or one of the two flight controllers, additionally use the ESC rev 1.0 board, flash the respective firmware to both, run the respective CubeIDE projects in debug mode, send a throttle value from the Flight Controller and make sure the PWM wave is received and translated correctly.

Electronic Speed Controller to Motor Communication

The path between the ESC and the brushless motor is the three phase commutation drive. Testing is done by spinning a motor via the ESC's command across the full range of throttle values and verifying voltage waveforms through an oscilloscope. As well as back EMF computations for RPM feedback. For successful validation, there will be a smooth startup, sustained RPM, and correct throttle commands without stalling, or overheating the MOSFETS.

5.4 System Testing

System level testing is achieved by combining the unit, interface, and integration tests described above into a complete end-to-end verification of the system. A passing system test requires that USB communication between the desktop application and flight controller is functioning correctly, that radio controller inputs are received and interpreted by the flight controller, that DShot signals are correctly generated and transmitted to the ESC, and that the ESC drives the motor as expected. Together these tests cover the full command path from user input to motor output and directly map to the core functional requirements of the system. CubeIDE debug mode, oscilloscope measurements, and the desktop application's live data log are the primary tools used throughout system testing.

5.5 Regression Testing

Hardware:

Each board revision is treated as a regression target against the previous revisions validated behavior. When a new revision is fabricated, the full unit and interface test sequence is rerun before any new functionality is tested. Continuity checks on power rails, isolated regulator validation through solder jumpers, SWD programmability, oscilloscope verification of communication protocols at the test points.

Firmware:

After we merge main into our branch with new features, we go through and test all the old functionality to ensure it is still functional. We have a list of peripherals and protocols to check that all functionality is still working, and if it is not, we immediately fix the broken communication in order to not lose work or progress.

Software:

As new features are added to the desktop application or existing functionality is modified, previously passing tests are re-run to confirm that no existing behavior has been broken. The critical features that must remain functional through any changes are the USB connection and communication with the flight controller, the accuracy of the live data log display, and the correct behavior of all implemented UI components. These are prioritized because they form the core of the desktop application's functionality and any regression in these areas would directly impact the usability of the software. Testing is performed manually by re-running the relevant UI walkthroughs and verifying live data against firmware responses using the desktop application's built-in data logging.

5.6 Acceptance Testing

Since the project is self-directed with no external client, acceptance testing is conducted by the development team against the defined system requirements. The team will demonstrate that each functional requirement is met by walking through the complete system, from establishing a desktop application connection to the flight controller, through radio input processing, DShot motor control output, and UI functionality. Non-functional requirements such as communication latency, data accuracy, and documentation usability are verified through live data logging, oscilloscope measurements, and external user feedback gathered during user testing. The system is considered accepted when all critical functional paths operate correctly and feedback from outside

testers confirms usability of the software and documentation.

5.7 User Testing

Hardware:

The primary testers are electrical or computer engineers attempting to reproduce or modify the design. Given the complexity of full board reproduction, realistic feedback comes from a small group of qualified reviewers. Primarily Gshash Mahmoud and our advisor Dr. Neihart. Who evaluates documentation usability and design clarity. Testing focuses on whether a user could swap or add a component (e.g., a different IMU or regulator) and understand the downstream impact on schematic, layout, and firmware integration from the published documentation alone. Feedback is gathered through direct review sessions and applied to the next documentation and board revision.

Firmware:

Testers will be asked outside of the group to use our product. From their feedback we will adjust certain available parameters. If the user requests different features to be available we will tailor the firmware around the ability to customize those parameters. Each parameter available has to be explicitly written in firmware to be customizable. The feedback will be collected through observations of the user experience and their spoken opinions.

Software:

The team will involve real users outside of the development group to evaluate the usability and overall user experience of the software. Testers will be asked to interact with the desktop application UI and provide feedback on ease of use, intuitiveness of the controls, and whether the available drone configurations meet their expectations or if additional options would be useful. Separately, testers will be asked to navigate and read through sections of the open-source documentation hosted on the GitHub Pages website, providing feedback on clarity, organization, and navigability. Feedback will be gathered through informal observations and open-ended responses, and will be used to identify areas for improvement in both the application and documentation before final delivery.

5.8 Results

Hardware Team:

Unit and interface testing have been completed successfully across all three boards. Continuity checks confirmed no power rail shorts, isolated regulator validation through solder jumpers verified each rail within tolerance, and SWD programmability was established on both the FC and ESC MCUs. Communication protocols (SPI, I^2C, UART) were verified at the test points with an oscilloscope USB 2.0 Full Speed communication between the FC and the desktop application is detected and exchanges packets reliably.

The remaining open item is closed-loop motor control on the ESC. Open-loop commutation has been demonstrated, and back-EMF sensing is currently being implemented.

Firmware:

For firmware we have established most of the sensor connections, DSHOT protocol, USB communication, USB data streaming, and external interaction with a controller. The DSHOT protocol we have verified to be 600 Khz with 26.7us frame size. This is required to meet our 8Khz PID loop that is a requirement for the drone. We have established accurate USB communication verified through testing with our custom desktop application.

Through that we have been able to interact with the drone and establish some adjustable settings. There are requirements for all of our users as they need to be able to interact and customize the drone. The USB streaming have been verified in our in desktop application through the live display of the drone orientation. The USB streaming exists to help verify sensor accuracy in the event it needs to be calibrated. Our sensors have been verified by comparing their data against expected data. For instance, our magnetometer data was compared against expected values for the region of Iowa. We have incorporated all of this functionality around an operating system.

Through the usage of the operating system we have learned a lot about priority scheduling and timing. Moving forward, next steps are to integrate this functionality with a control system. The control system will utilize the data from sensors to ensure we can achieve steady level flight. We need to ensure that priorities in the operating system do not hinder the performance of the motor positioning logic.

Software:

On the software side, USB communication between the desktop application and the flight controller has been verified, with the application successfully establishing a connection, transmitting commands, and receiving responses with accurate data displayed in real time through the live data log. Unit testing of the UI has been completed for the currently implemented components, with buttons, inputs, and status displays confirmed to behave as expected. Testing of the drone configuration options, gyroscopic view, sensor calibration, and power and battery displays is ongoing as those features continue to be developed. User feedback on the UI usability and documentation will be gathered as the project approaches completion.

6 Implementation

Hardware:

The hardware implementation consists of four separate subsystems, the drone frame, the flight controller, the electronic speed controller, and the GNSS board.

Drone Frame:

The drone frame was implemented as a custom 3D printed design to host the OSCAR flight electronics. Commercially available frames only support 20×20mm or 30×30mm board mounts, which are incompatible with the 50×50mm footprint of our PCBs, so a custom frame was modeled in Onshape and printed at the Student Innovation Center.

Following the honeycomb and skeletonized approach demonstrated by King Abdulaziz University [2], weight was reduced while preserving rigidity. The center plate carries the 50×50mm M2 mounting pattern for the FC and ESC stack, and four arms extend outward to clear the 5-inch propellers and terminate in M3 motor mounts. The completed frame was printed in PLA and weighs 63 grams.

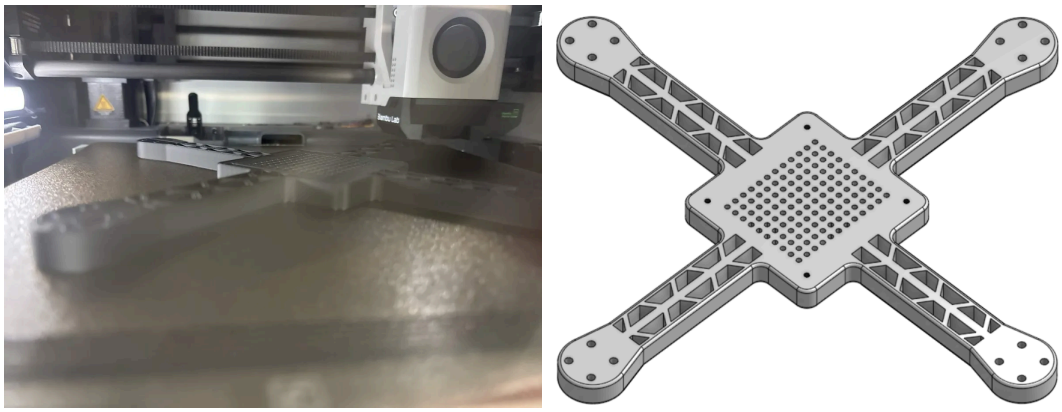


Figure 17. 3D printed frame for the drone.

Flight Controller:

The FC was designed in KiCad, an open-source PCB tool, as a 4-layer board fabricated by JLCPCB within their standard manufacturing constraints. Assembly was performed by the team using a stencil and reflow oven; this was required for the IMU and barometer, both of which use LGA packages that are impractical to hand solder and instead rely on reflow or hot air rework.

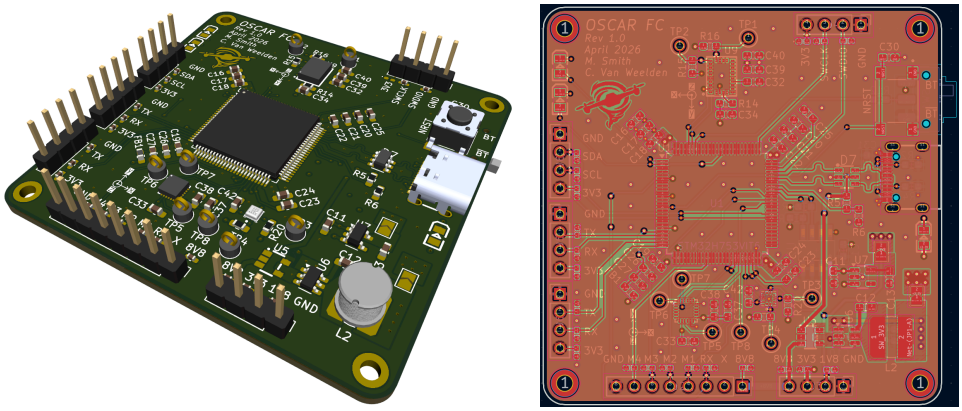


Figure 18. 3-D rendering and layout of the flight controller.

ESC:

The ESC was designed in KiCad as a 4-layer board fabricated by JLCPCB. The tight layout space made the use of small 0402 sized components needed for this first design. Due to our inexperience with reflow ovens and high-density assembly, we designed around hand soldering the entire backside of the board and using a solder stencil for the top. This left a lot of room on the back but a very cramped top layer. For future designs, we will look into replacing as many 0402 sized components as possible with 0603 or larger and using the backside more heavily.

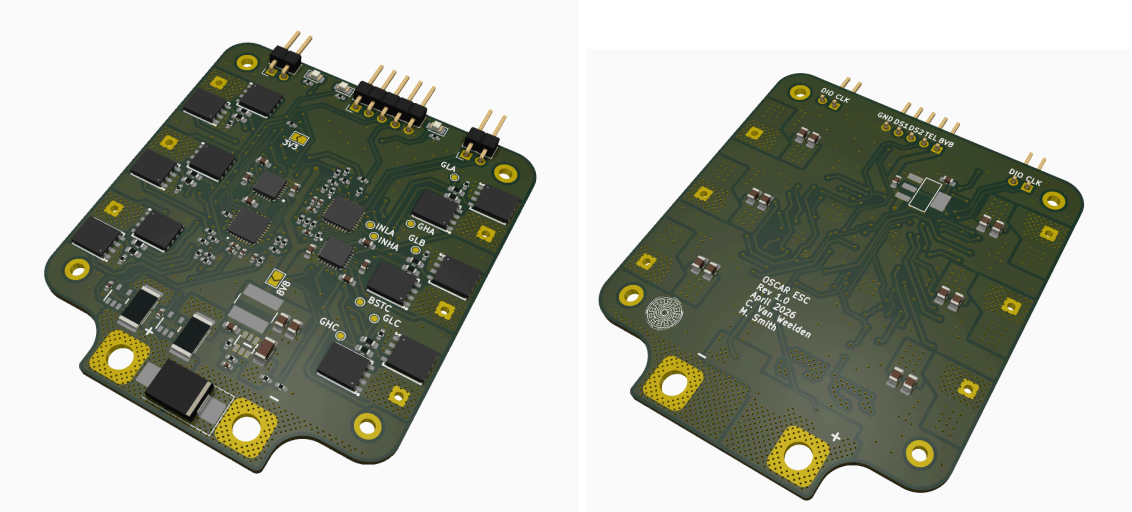


Figure 19. 3-D rendering of the electronic speed controller.

The use of large copper pours and thermal stitching vias was chosen to help conduct heat away from the components and into the air. 2oz copper was used for all internal and external copper layers to double the cross sectional area of these copper pours. This was done to reduce the resistive power losses from the high current and help increase thermal conductivity away from high heat generating components like the MOSFETs.

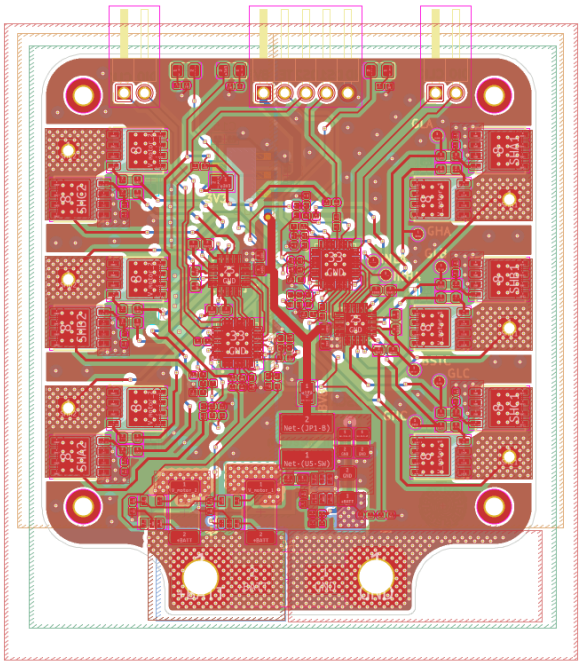


Figure 20. Layout of the electronic speed controller.

GNSS:

The GNSS was designed in KiCad as a 4-layer board fabricated by JLCPCB within their standard manufacturing constraints. Assembly was performed by the team using a stencil and reflow oven; this was more forgiving than the FC since the only difficult part was the magnetometer's LGA package. The passive patch antenna was mounted on the opposite side of the board as required by its datasheet, which calls for an uninterrupted ground plane beneath it.

The RF trace from the antenna feed to the GNSS module was implemented as a 50Ω impedance-matched transmission line, with trace width and clearance calculated against the board stackup to match the antenna and module input impedance. Stitching vias were placed along both sides of the RF trace to form a via fence, tying the surrounding ground plane together and suppressing ground loops and stray coupling that would otherwise degrade RF performance.

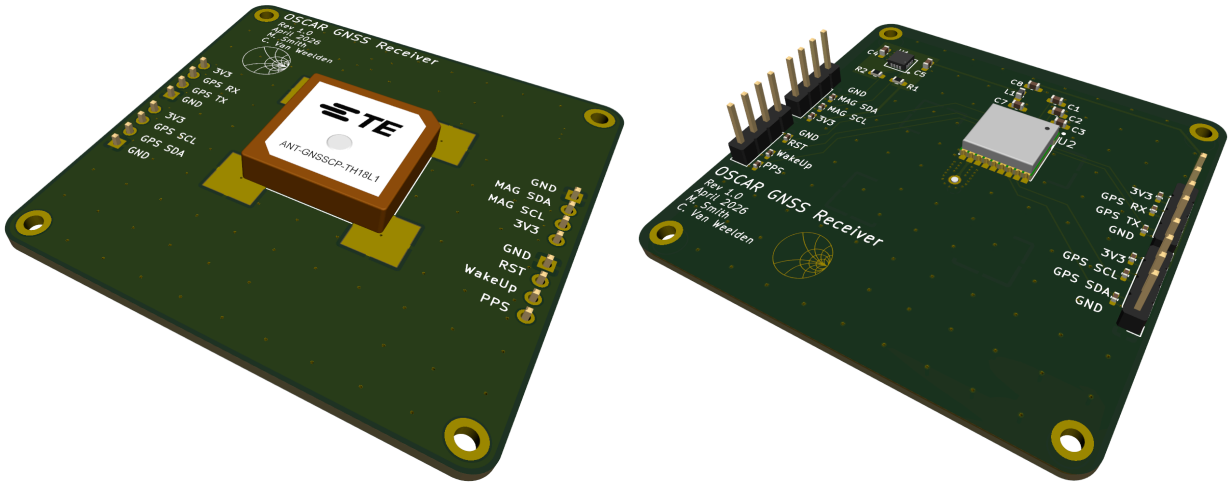


Figure 21. 3-D rendering of the GNSS board.

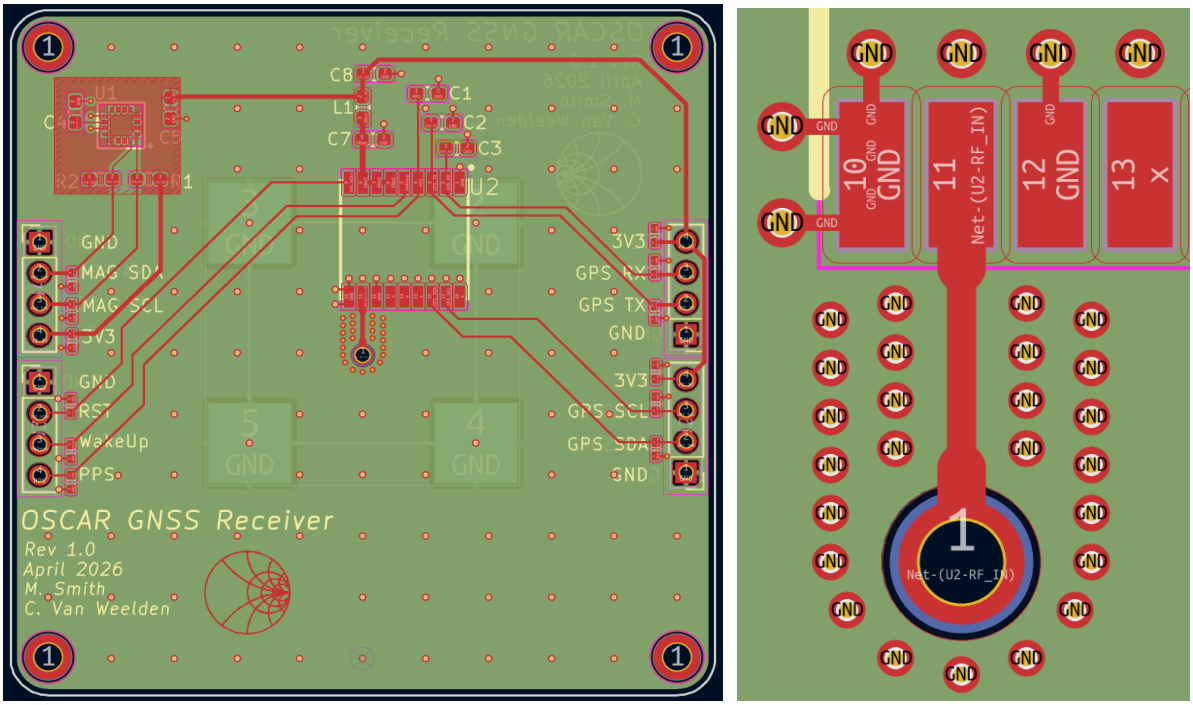


Figure 22. Layout of the GNSS board.

Firmware:

The firmware implementation consists of four primary subsystems: the real-time operating system architecture, sensor configuration and data acquisition, radio communication integration via ELRS, and motor control through DSHOT protocol. These components

collectively enable deterministic control, reliable state estimation, and real-time responsiveness of the flight controller.

Real-Time Operating System (FreeRTOS):

The firmware is built on a FreeRTOS kernel to provide deterministic scheduling and modular task management. System functionality is divided into multiple concurrent tasks, including sensor polling, communication handling, control loop execution, and telemetry streaming. Task prioritization is configured to ensure no tasks experience starvation, but important tasks such as communication and control can preempt other menial tasks. Inter-task communication is implemented using queues, semaphores and mutexes to maintain data integrity and prevent race conditions.

Sensor Configuration

Firmware configuration has been implemented for the IMU, barometer, and magnetometer. Each sensor is initialized through its respective communication interface (I2C/SPI). The IMU provides high-frequency accelerometer and gyroscope data. Sensor polling tasks continuously get raw data, which is then processed and made available for other tasks.

Radio Communication

The firmware integrates LoRa using an external module based on ExpressLRS (ELRS). Communication between the flight controller and the ELRS module is handled via a UART interface configured for low-latency, high frequency packed reception. Incoming control packets are parsed to find the sync bit, and then stores the data into direct memory to be unpacked into channel data. This allows for a reliable and responsive control link between the pilot and the drone.

Motor Control via DSHOT Protocol:

Motor control is implemented using a unidirectional version of DSHOT protocol, allowing for precise and low latency communication between the FC and ESCs. Digital throttle commands are generated within the control loop and transmitted as timed pulse sequences over dedicated GPIO or timer peripherals. The motor outputs have been validated through oscilloscope measurements and function testing with ESC Hardware.

Software:

The software implementation consists of four primary deliverables: USB communication with the flight controller, the OSCAR Ground Station desktop application, a Windows executable installer, and the OSCAR Docs open-source documentation website.

USB Communication & COM Port Connection:

The USB communication layer has been implemented using a native C module compiled into the Electron application via Node-API. The module uses the libserialport library to interface directly with the operating system's COM port drivers. Upon launch, the application scans all available COM ports and initiates a handshake sequence to identify and verify the correct flight controller connection. Once a valid connection is established,

the backend constructs binary communication packets consisting of a 2-byte command header, a 4-byte data length field, up to 56 bytes of raw payload data, and a 2-byte CRC-16/CCITT checksum. Every response received from the flight controller is validated against its checksum before being parsed and forwarded to the frontend. A secondary COM port has also been implemented to handle continuous data streaming independently of standard request-response communication, ensuring that streaming does not block or delay command handling. Both COM ports have been verified to be functional through live data logging and manual testing against the flight controller.

Desktop Application UI Pages:

The OSCAR Ground Station desktop application has been implemented using the Electron framework with a frontend built in JavaScript, HTML, and CSS. All planned UI pages have been constructed, and several have been fully integrated with USB communication. The pages currently populated with live flight controller data include the Setup & Ports page, Configuration page, Power & Battery page, and Motors page. These pages send requests to the backend upon load, receive structured JSON responses from the flight controller, and display the returned values in real time. The remaining pages (Home, PID Tuning, Receiver, and GPS) have their UI bases fully built and are pending full USB data integration. A gyroscopic 3D visualization of the drone has also been implemented using Three.js, which maps incoming quaternion telemetry data from the IMU to a real-time 3D rendered drone model in the viewport. This allows users to visually verify drone orientation during pre-flight setup.

Executable Installer:

The desktop application has been packaged into a Windows executable installer using Electron Forge and the Squirrel installer framework. The installer handles application setup, file placement, and desktop shortcut creation on the user's machine. The installer has been tested and correctly installs and launches the application on windows devices. The executable will be available on the open-source documentation website for download.

Open-Source Documentation Website:

The OSCAR Docs website has been implemented using GitHub Pages and is publicly accessible through the project's Senior Design website. The site structure and template have been established, and a decent amount of documentation has been put on it. The documentation website is intended to serve as the primary reference for anyone looking to replicate, modify, or learn from the OSCAR system, covering hardware, firmware, and software components. Document population for each subgroup is ongoing and will continue to be added as the project progresses towards the end of next semester.

7 Ethics and Professional Responsibility

Ethical responsibility is more than just technical performance. It means risk mitigation, accountability, and following applicable regulations. Our team is proactively committed to ethical decision-making and emphasizes safety as a primary design constraint.

Professionally, we define responsibility as maintaining clear communication, documentation, and accountability throughout the design process. Each team member is expected to communicate clearly and uphold integrity in reporting results, regardless of a positive or negative outcome.

7.1 Areas of Professional Responsibility/Codes of Ethics

Table 9. IEEE ethics principles mapped to project implementation and engineering practices.

Area	Definition	Relevant IEEE Item	Project Application
Technical Competence & Safety	Ensuring our work is accurate, and reliable, while trying to minimize risks associated with the system.	IEEE #1: "To hold paramount the safety, health and welfare of the public"	Our team ensures technical correctness through validated sensor integration (IMU, barometer, magnetometer) and real-time scheduling using FREERTOS. We test firmware modules individually and in integration to ensure reliable operation. We address safety through stable control timing, and careful motor signal generation via DSHOT.
Honesty & Integrity	Being truthful about the system capabilities, limitations, and results.	IEEE #3: "To be honest and realistic in stating claims or estimates"	Our documentation and desktop application accurately reflect system status and limitations. Such as incomplete or still in progress features.
Respect for Others	Considering users and teammates in design and collaboration.	IEEE #7 "To treat all persons fairly and with respect"	The project has three different subteams, which requires clear communication and respect for integration dependencies.
Lifelong Learning	Continuously improving skills and adapting to new challenges.	IEEE #5 "To improve the understanding of technology... and its consequences"	The team has learned new technologies (FreeRTOS, ELRS, embedded protocols) and applied them throughout the project.
Professional Practice	Following Standards, documentation , and ethical engineering practices.	IEEE #6 "To maintain and improve our technical competence"	We follow structured development processes including modular firmware design, documentation via OSCAR Docs, and version-controlled codebases.

Reflection:

Area of Strength:

Our team is performing strongly in technical competence. This is demonstrated through the successful integration of multiple complex subsystems, including real-time task scheduling with FreeRTOS, sensor data acquisition (IMU, barometer, magnetometer), and communication protocols like ELRS and DSHOT. Each subsystem has been individually validated and then integrated into a cohesive system. This reflects strong adherence to IEEE ethical principles by ensuring that the system is reliable and functions as intended, which directly supports user safety and system effectiveness.

Area for improvement:

One area where our team can improve is in addressing the broader societal and environmental impacts. While we acknowledge there is risk of misuse, and environmental concerns, our design does not actively mitigate these issues. Moving forward the team should incorporate clear usage guidelines, and consider implementing safety restrictions wherever possible. Improving in this area would better align the project with ethical responsibilities beyond purely technical performance.

7.2 Four Principles

Table 10. IEEE Code of Ethics principles applied to UAV system development and engineering practice.

Area	Beneficence	Nonmaleficence	Respect for Autonomy	Justice
Public health, safety, and welfare	Enables potential for advancements in research	OSCAR poses inherent crash, weaponization, privacy, and surveillance risks common to any drone platform.	Users can customize OSCARS's hardware, firmware, and software for their own use cases.	Open-source access allows equal opportunity for students and researchers to benefit.
Global, cultural, and social	Promotes global collaboration and knowledge sharing through open source design and documentation.	Potential misuse across different regulatory or cultural environments where drone usage norms differ.	OSCAR is completely open source. This provides individuals to make their own decisions about how to alter our product. They add or remove whatever is necessary to fit their use case.	Freely available design reduces barriers for underrepresented or resource-limited communities to participate in drone development

Environmental	Can be used for environmental monitoring, research, and data collection	Environmental impact from electronic waste, battery disposal, and resource consumption during manufacturing.	Autonomy framework is human centered which ensures operators maintain control over how and where the drone is used.	Open design enables reuse, repair, and modification, reducing waste compared to closed systems.
Economic	Provides an alternative to commercial drones, supporting education and small-scale development.	OSCAR is not optimized for low cost, the full BOM is more expensive than commercial hobby flight controllers. Builders take a more financial risk from crashes or board failures	OSCAR is completely customizable allowing individuals to adjust the product based on a budget.	The high BOM cost, and US-focused part selection means OSCAR is most accessible to users with more financial resources and is US based.

The most important broader context principle pair for OSCAR is Public Health, Safety, and Respect for Autonomy. Releasing the full hardware, firmware, and software as open source gives users complete control to modify, extend, or strip down the platform for their own use cases. We ensure this benefit by maintaining a public repository with complete schematics, gerber files, bill of materials, firmware and software code as open source gives users complete control to modify, extend, or strip down the platform for their own use cases. We ensure this benefit by maintaining a public repository with complete schematics, PCB files, firmware, and documentation written for someone else learning the platform rather than existing experts.

OSCAR is most lacking in the areas of Economic, Justice, and Nonmaleficence. The BOM cost is higher than that of commercially available flight stacks, and the US-focused supply chain limits the design's accessibility to builders globally. Separately, the team's open nature has no mechanisms to prevent downstream use. These negatives are partially offset by the strong Autonomy and open access benefits described above. The same openness that creates misuse risks is what enables legitimate research, education, and customization. To improve directly, the team could publish a lower-cost variant BOM with cheaper component alternatives that are also globally available.

7.3 Virtues

Team Virtues:

1. Reliability
    a. Reliability is the capacity to deliver upon intended functionality with consistency. Our team has demonstrated reliability in many ways this semester. All members of our team have completed the tasks they promised to complete within the timeframe they promised to complete them in. Each sub-team depends on each other. For example, the software team depended upon the firmware team in order

for USB communication development. Additionally, the firmware team depended on the hardware team in order to develop and test their work. At the end of this semester, we reached the goal we had set for ourselves, further proving our reliability and consistency as a team.

2. Transparency
    a. Transparency is the practice of being open, honest, and accountable. Our team has demonstrated transparency in our commitment to being open-source via documenting our design decision, our process developing and testing our final design, and uploading this information to a publicly available website.
3. Resilience
    a. Resilience is the ability to persevere and recover from challenges and obstacles. Our team has persisted through several adversities throughout the semester. The design and testing processes have never been on easy street. Bumps and setbacks are a huge part of the design process, and our team has handled them with determination and grace. We take all the setbacks with stride, and rather than wasting time placing blame or arguing, we accept that an issue occurred and immediately move on to fixing the issue.

Individual Virtues:

- **Matthew Smith**
    - Demonstrated: Humility
        - Humility is important to me because no single person on a seven-person interdisciplinary team has all the answers, and pretending otherwise leads to worse design decisions. I've demonstrated this by accepting feedback on wrong decisions. As a team lead, I've made mistakes by delegating work to a subteam without discussing or understanding where they were in their workload or their input on the task. Acknowledging one's mistakes leads to better outcomes for everyone on the team.
    - Not Demonstrated: Silence
        - Silence, as Franklin defines it, is speaking only what benefits others or yourself and avoiding trifling conversation. This matters to me because team meeting time is limited, and every minute spent on tangents delays actual design work. I haven't demonstrated this well; I have tended to go into tangents and discussions of future problems, rather than directly face the problem right in front of us. To improve, I plan to be more aware when my attention drifts off topic and to save other problems for a later one-on-one's or future meeting rather than the planned full-team meeting.
- **Collin Van Weelden**
    - Demonstrated: Tranquility
        - Tranquility is important to me and my interactions within this team. Mistakes and disagreements can feel like personal attacks if expectations are not met. But lashing out or building resentment will only serve to make the issue larger and push the team away. Patience and communicating with the underlying understanding that everyone on the team is moving towards the same goal can reduce any harmful reactions such as personal attacks towards others on the team. When faced with

disagreements within the team, I can respect the choices of my teammates and trust that the choices they make are aligned with the team.
    - Not Demonstrated: Silence
        - Silence is an important virtue to me that I see myself lacking in this project. Silence to me means avoiding unnecessary conversations that take away important time from the team. As part of a team of engineers who are also friends, it becomes hard to avoid getting off topic. A virtue that I am missing is the ability to separate my friendly conversation from productive project conversations. Off topic conversations make time management much more challenging, not just for myself but the other members of the team as well.
- **Paul Zell**
    - Demonstrated: Patience
        - Patience is important to me because everyone comes from a different background. It is important to take the time to understand that and be as helpful as possible. I have demonstrated this by being as helpful as possible and answering as many questions as it takes to make sure an understanding is achieved.
    - Not Demonstrated: Optimism
        - Optimism is important because it helps to keep morale up. Keeping an outward joyful outlook on things keeps productivity high as well. While this semester I have not been the most optimistic when it comes to meeting deadlines I can definitely work on that. To try and demonstrate this I can change my language when expressing concerns and attempt to give everything a more positive spin. I can also try to encourage others to keep a positive outlook on things.
- **Chloe Atwood**
    - Demonstrated: Reliability
        - Reliability is important to me because trust is the foundation of effective teamwork. I demonstrated this by consistently maintaining and updating our Desktop Application and Open-Source documentation, and by always delivering on my commitments when I said I would.
    - Not Demonstrated: Patience
        - Patience is something I seem to struggle with. I do not like to wait for things and that sometimes makes me frustrated. When working on the Desktop Application I would often get antsy when bugs fixes or new features took longer than expected to implement.
- **Jessica Reiff**
    - Demonstrated: Reliability
        - Reliability is essential in team settings because it builds trust and accountability among team members. When individuals consistently meet deadlines, communicate effectively, and follow through on commitments, the entire team can function more efficiently and with greater confidence. Tasks are often interdependent, one person's reliability directly impacts the progress of others. Without it, delays and miscommunication can disrupt the project timeline and success.
    - Not Demonstrated : Open-Mindness

        - Sometimes I have a bit of an idea of how I think something should go, and I have trouble listening to teammates with an open mind and really thinking about their point of view.
- **Gracie Finger**
    - Demonstrated: Compassion
        - Compassion is a very important virtue to me. As a Christian, I am called to love the people around me in the same way Christ has loved me. This looks like having compassion on people in every situation I find myself in. In an engineering design context, this compassion is expressed in many different ways. I have demonstrated it during this project in the ways that I have made myself available to my teammates, whether it be emotional, technical, or general support.
    - Not Demonstrated: Curiosity
        - Curiosity is important to me because it allows me to further appreciate the world and how it works. This would be helpful to our project as it would allow me to have more innovative ideas to solve problems. I have lacked curiosity this semester because I have been so focused on completing the tasks at hand that I have not found curiosity worth the time. I can improve in demonstrating this virtue by asking more questions and allowing myself the time to dive deeper into concepts.
- **Grace White**
    - Demonstrated: Sincerity
        - Sincerity is a key virtue in my life. I strive to never regret my actions, even in err, because they always teach me something. When I speak, it is very rarely without thought. This intentionality has shaped how I show up in this project in more ways than one. It has helped me not to mistake miscommunication for malicious acts. Beyond conflict, it has also allowed me to connect more genuinely with my group members. When people can tell you mean what you say, trust comes more naturally. That trust has made collaboration smoother and conversations more honest throughout this project.
    - Not Demonstrated: Thorough Documentation
        - Thorough documentation is a key aspect of our project, as being open source is the backbone of what we are building. Others being able to pick up where we left off depends on the clarity and completeness of what we leave behind. However, I have a tendency to put my head down, get the work done, and miss the detailed work past getting a functional product. I am someone who moves towards the finish line, and in doing so, the smaller steps can slip through the cracks. My code could be better commented, and I am sure there are small steps that I forget to record as I do them subconsciously now. Recognizing this is something I am actively working on, because I understand that in an open source project, an undocumented solution can be just as limiting as no solution at all.

8 Closing Material

8.1 Conclusion

Our goals are to create a drone that can achieve steady level flight while having all the designs available publicly. So far we have achieved the creation of the initial prototype of the drone. While it does not fly yet, it does contain much of the needed functionality. Below is a list of everything we have accomplished towards our goal:

- Designed and assembled three custom first iteration PCB designs
    - Flight Controller
    - Electronic Speed Controller
    - Global Navigation Satellite System receiver
- Built firmware utilizing FreeRTOS with verified DSHOT communication at 600kHz
- Dual COM port USB communication
- Sensors (IMU, magnetometer, barometer) initialized
- Desktop application with most UI pages built and several pages integrated with live data
- Desktop application can be installed on windows devices as an executable
- Live open source documentation website hosted by GitHub Pages

For pacing, our goal this semester was to be able to spin a motor using the communication from the desktop application. We were able to achieve this goal through diligent efforts from all the members. Our plan of action for achieving the end product is to get full motor control as soon as possible and begin our work on controls. While we will likely have another board revision during this time those two main concepts are our main limiting factors. Each group member is dedicated to achieving these goals and we feel we can accomplish them.

8.2 References

[1] F. Kateb *et al.*, “Drone Frame Optimization via Simulation and 3D Printing,” *Computers*, vol. 14, no. 8, p. 328, Aug. 2025, doi: https://doi.org/10.3390/computers14080328.

[2] Y. Kim, A. Shoufan, M. Alattar, and R. Damseh, "Challenges in Dynamic Analysis of Drone Firmware and Its Solutions," IEEE Access, vol. 12, pp. 95301–95320, 2024, doi: 10.1109/ACCESS.2024.3425604.

[3] H. Ha, *TinyUSB*, v0.20.0. GitHub, 2025. [Online]. Available: https://github.com/hathach/tinyusb

[4] D. Gamble, *cJSON*, v1.7.18. GitHub, 2024. [Online]. Available: https://github.com/DaveGamble/cJSON

[5] STMicroelectronics, “High-performance and DSP with DP-FPU, Arm Cortex-M7 MCU with 2 Mbytes of Flash memory, 1 MB RAM, 480 MHz CPU, Art Accelerator, L1 cache, external memory interface, ISP, and cryptographic acceleration,” STM32H753xI datasheet, DS12117 Rev 9, Oct. 2021.

[6] BINARYUPDATES, “How to use FreeRTOS with STM32 Microcontroller–Example Project,” YouTube, Mar. 19, 2023, [Online Video]. Available: https://www.youtube.com/watch?v=hArOFZlov9c. Accessed: May 1, 2026.

[7] A. Haber, “Generate PWM Signals in STM32 Microcontrollers (STM32CubeIDE and HAL),” *Aleksandar Haber: Control Systems, Robotics, and Machine Learning*, Dec. 15, 2022. [Online]. Available: https://aleksandarhaber.com/generate-pwm-signals-in-stm32-microcontrollers/. Accessed: May 1, 2026.

[8] mokhwasomssi, “stm32_hal_dshot: STM32 HAL DShot Driver,” GitHub repository, Feb. 15, 2024. [Online]. Available: https://github.com/mokhwasomssi/stm32_hal_dshot. Accessed: May 1, 2026.

[9] Betaflight, “Dshot: Bidirectional Dshot,” *Betaflight Documentation*, 2024. [Online]. Available: https://www.betaflight.com/docs/development/API/Dshot#bidirectional-dshot. Accessed: May 1, 2026.

[10] The Things Network, “What is LoRaWAN?,” *The Things Network Documentation*. [Online]. Available: https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/. Accessed: May 1, 2026.

[11] O. Liang, “A Complete Guide to Flashing and Setting Up ExpressLRS,” *Oscar Liang*, Mar. 6, 2024. [Online]. Available: https://oscarliang.com/setup-expresslrs-2-4ghz/. Accessed: May 1, 2026.

[12] ExpressLRS, “ExpressLRS Source Code,” GitHub repository. [Online]. Available: https://github.com/ExpressLRS/ExpressLRS. Accessed: May 1, 2026.

[13] stepinside, “Arduino-CRSF: Arduino-CRSF.cpp,” GitHub repository. [Online]. Available: https://github.com/stepinside/Arduino-CRSF/blob/main/src/Arduino-CRSF.cpp. Accessed: May 1, 2026.

[14] ExpressLRS, “Happymodel EP1/EP2 RX,” *ExpressLRS Quick Start Guide*. [Online]. Available: https://www.expresslrs.org/quick-start/receivers/hmep2400/. Accessed: May 1, 2026.

[15] CRSF-WG, “CRSF Protocol Wiki,” GitHub repository wiki. [Online]. Available: https://github.com/crsf-wg/crsf/wiki. Accessed: May 1, 2026.

[16] PX4 Autopilot, “CRSF Telemetry,” *PX4 User Guide*. [Online]. Available: https://docs.px4.io/v1.14/en/telemetry/crsf_telemetry.html. Accessed: May 1, 2026.

[17] CRSF-WG, “Packet Types,” GitHub repository wiki. [Online]. Available: https://github.com/crsf-wg/crsf/wiki/Packet-Types. Accessed: May 1, 2026.

[18] CRSF-WG, “CRSF_FRAMETYPE_LINK_STATISTICS,” GitHub repository wiki. [Online]. Available: https://github.com/crsf-wg/crsf/wiki/CRSF_FRAMETYPE_LINK_STATISTICS. Accessed: May 1, 2026.

[19] sigrok, "libserialport API Documentation," sigrok.org. [Online]. Available: https://sigrok.org/api/libserialport/unstable/. Accessed: May 2, 2026.

[20] Node.js, "C/C++ addons with Node-API," Node.js Documentation. [Online]. Available: https://nodejs.org/api/n-api.html. Accessed: May 2, 2026.

[21] sigrok, "Libserialport," sigrok.org wiki. [Online]. Available: https://sigrok.org/wiki/Libserialport. Accessed: May 2, 2026.

[22] sigrokproject, "libserialport," GitHub repository. [Online]. Available: https://github.com/sigrokproject/libserialport. Accessed: May 2, 2026.

[23] Electron, "Application Packaging," Electron Documentation. [Online]. Available: https://www.electronjs.org/docs/latest/tutorial/application-distribution. Accessed: May 2, 2026.

[24] Electron Forge, "Custom App Icons," Electron Forge Documentation. [Online]. Available: https://www.electronforge.io/guides/create-and-add-icons. Accessed: May 2, 2026.

[25] Electron Forge, "Getting Started," Electron Forge Documentation. [Online]. Available: https://www.electronforge.io/. Accessed: May 2, 2026.

[26] Electron Forge, "Importing an Existing Project," Electron Forge Documentation. [Online]. Available: https://www.electronforge.io/import-existing-project. Accessed: May 2, 2026.

[27] Electron Forge, "Squirrel.Windows," Electron Forge Documentation. [Online]. Available: https://www.electronforge.io/config/makers/squirrel.windows. Accessed: May 2, 2026.

[28] three.js, "Quaternion," three.js Documentation. [Online]. Available: https://threejs.org/docs/#Quaternion. Accessed: May 2, 2026.

[29] three.js, "QuaternionKeyframeTrack," three.js Documentation. [Online]. Available: https://threejs.org/docs/#QuaternionKeyframeTrack. Accessed: May 2, 2026.

[30] Betaflight, "Betaflight App," Betaflight Wiki. [Online]. Available: https://betaflight.com/docs/wiki/app. Accessed: May 2, 2026.

[31] “32-bit 4-1 MK 1.1 - Platform for creating and sharing projects - OSHWLab,” *Oshwlab.com*, 2024. https://oshwlab.com/hawaii0707/32wei-si-ge-yi-dian-dia0-mk1.1-1 (accessed May 03, 2026).

[32] “Hardware design for AM32 ESCs - The Devana Project,” *The Devana Project* -, Nov. 30, 2025. https://thedevanaproject.com/2025/11/30/hardware-design-for-am32-escs/ (accessed May 03, 2026).

[33] “Hardware Design | AM32,” *Am32.ca*, 2026. https://wiki.am32.ca/development/Hardware-Design.html (accessed May 03, 2026)

[34] “AN2009 APPLICATION NOTE PWM MANAGEMENT FOR 3-PHASE BLDC MOTOR DRIVES USING THE ST7MC.”

8.3 Appendices

Our project supports an external website that holds our implementations in more depth. This website is to serve as the main hub of our open source information. The link is as follows: https://gracefinger.github.io/OSCAR_Documentation.github.io/index.html

9 Team

9.1 Team Members

- Matthew Smith
- Collin Van Weelden
- Jessica Reiff
- Paul Zell
- Grace White
- Chloe Atwood
- Gracie Finger

9.2 Required Skill Sets for Your Project

Hardware:

- Soldering
- Reading Datasheets
- Picking parts relevant to the design
- PCB Design
- 3D modeling and printing experience
- Controls and PID
- Drone pilot experience

Firmware:

- Oscilloscope tracing
- Real time operating system priority decisions
- Capturing pinouts
- Debugging code
- Sending and packet sniffing USB communications
- Setting up PWM signals
- Continuous micro decision making
- Setting up interrupts
- Using tasks, mutexes, queues, and semaphores
- Reading and analyzing datasheets for the various hardware components
- Developing various types of communication protocols

Software:

- Frontend and Backend development of a Desktop Application
- Understanding of process isolation, multi-threading and utilizing locks
- Git version control
- USB Communication from a Desktop Application to a Firmware device
- Debugging code
- Ability to learn new information dynamically, quickly, and individually
- Web design to create and open-source documentation website
- Creating an installer and executable file for the Desktop Application

9.3 Skill Sets covered by the Team

Hardware:

- Matthew Smith - Hardware Designer, EE:
    - PCB design
    - PCB manufacturing
    - PCB assembly
    - Understanding of embedded system design
    - High Frequency PCB design

- Collin Van Weelden - Hardware Designer, EE:
    - PCB design
    - PCB manufacturing
    - Understanding of embedded system design
    - Circuit design and analysis
    - Hardware troubleshooting
    - Knowledge of existing FPV drones and drone components

Firmware:

- Jessica Reiff - Controls Lead & Misc Embedded, EE:
    - Debugging Code
    - Reading and analyzing datasheets
    - Writing embedded C code
- Paul Zell - Embedded OS & Embedded Lead, CPR E:
    - Writing embedded C code
    - Debugging operating system priorities
    - Code integration
    - Debugging code
    - Establishing communication protocols
- Grace White - Embedded Communications Lead, CPR E:
    - Clear communication of needs and wants to the other teams
    - Writing embedded C code
    - Debugging code
    - Git merge capabilities
    - Careful manipulation of auto generated files
    - Effective use of semaphores, tasks, queues, and mutexs with their associated priorities
    - Establishing and customizing communication protocols
    - Cross protocol trigger handling

Software:

- Chloe Atwood - Git Master & Frontend Software Dev, SE:
    - Application Development
    - Understanding of process isolation, multi-threading and utilizing locks
    - Git version control

    - Debugging code
    - Setting up USB Communication from a Desktop Application
    - Ability to learn new information dynamically, quickly, and individually
    - Web design and development
    - Creating an installer and executable file for the Desktop Application
- Gracie Finger - Backend Software Dev, CPR E:
    - Application Development
    - Git version control
    - Debugging code
    - Ability to learn new information dynamically, quickly, and individually
    - Web design and development
    - Creating an installer and executable file for the Desktop Application

9.4 Project Management Style Adopted by the team

For our senior design project, we are using a hybrid approach that combines elements of both waterfall and agile methodologies. Our process follows a waterfall structure in that we prioritize and develop the system in stages, starting with the most critical components before moving on to less essential features. At the same time, we incorporate agile practices by working in short, deadline-driven cycles where each team member is responsible for implementing, testing, and debugging their assigned features. This allows us to maintain overall structure while still adapting and making continuous progress throughout development.

9.5 Initial Project Management Roles

- Matthew Smith – Team Leader
- Collin Van Weelden – Hardware Lead
- Jessica Reiff – Controls Lead
- Paul Zell – Embedded Systems Lead
- Grace White – Embedded Communications Lead
- Chloe Atwood – Software & Version Control Lead
- Gracie Finger – Software & Embedded Integration Lead

9.6 Team Contract

Team Members:

1)	Grace Finger	2)	Jessica Reiff
3)	Chloe Atwood	4)	Grace White
5)	Paul Zell	6)	Collin Van Weelden
7)	Matthew Smith		

Team Procedures

Day, time, and location (face-to-face or virtual) for regular team meetings:

Large Group Team Meetings: Thursdays at 3:00 - 4:00

- Hardware submeetings: Wednesday 2pm-4pm & Friday 1pm-5pm.
- Software/firmware submeetings: Mondays 4:15pm - 5:15pm
- Software submeetings: Wednesdays 1pm-3pm

2. Preferred method of communication updates, reminders, issues, and scheduling:

Discord

3. Decision-making policy:

Consensus

4. Procedures for record keeping :

All meeting minutes will be kept in a shared Google Drive folder.

Large Group Meetings: Gracie will keep meeting minutes

Team (SW, HW, FW) Meetings:

- SW: Chloe will keep meeting minutes
- FW: Paul will keep meeting minutes
- HW: Matthew will keep meeting minutes

Participation Expectations

1. Expected individual attendance, punctuality, and participation at all team meetings:

The expectation is that when able everyone should attend the meetings on time that apply to them. Whether that be the large group meeting or a sub-group meeting. If an individual is unable to attend a meeting or will be late they are expected to reach out to the group and let everyone know. They are also expected to inquire about what was discussed at the meeting they missed. Each individual is expected to be coherent and active during the meetings. This includes but is not limited to asking questions, seeking clarification, active listening, etc.

2. Expected level of responsibility for fulfilling team assignments, timelines, and deadlines:

The expectation is for each individual to try their best to complete everything by the time decided by the group. If for any reason you are unable to meet a deadline, communicate to all team members as soon as you think you may not meet a deadline.

3. Expected level of communication with other team members:

All team members will be expected to communicate to the best of their ability. Everyone will be expected to treat all other members with respect.

4. Expected level of commitment to team decisions and tasks:

The expectation is that each individual member contributes the maximum amount of commitment that they are capable of given our course commitments.

Leadership

1. Leadership roles for each team member (e.g., team organization, client interaction, individual component design, testing, etc.):

- Matthew Smith: Team Leader
- Collin Van Weelden: Hardware Lead
- Jessica Reiff: Controls Lead
- Paul Zell: Embedded Systems Lead
- Grace White: Embedded Communications Lead
- Chloe Atwood: Software & Version Control Lead
- Gracie Finger: Software & Embedded Integration Lead

2. Strategies for supporting and guiding the work of all team members:

The expectation is that sub-teams will check in weekly either via Discord or in-person for status reports. During this time, potential roadblocks and questions will be discussed. Members are expected to maintain an open and encouraging attitude towards each other's work and abilities.

3. Strategies for recognizing the contributions of all team members:

We will come up with arbitrary awards to give each other at the end of the project. This may result in an award show that can be present on LinkedIn.

Collaboration and Inclusion

1. Describe the skills, expertise, and unique perspectives each team member brings to the

Team.

- Matthew Smith, EE,
    - PCB Schematic and Design (EE2300, EE3330, ETG).
    - RF/Microwave Design(EE5140).
    - Embedded systems experience at CNH Industrial
    - Controls work at John Deere.
    - Experience flying and building recreational drones.
- Chloe Atwood, SE:
    - Application Development - Class Projects, Research Assistant (2 full years, including summers)

    - Version Control - Class Projects, Research Assistant (2 full years, including summers)
- Paul Zell, CPR E:
    - Software preference in embedded programming. Has experience in application development as well.
    - Has industry experience (internship) developing embedded code to support microcontroller-sensor interactions.
- Gracie Finger, CPRE:
    - Application Development (class projects, software engineering internship)
    - Embedded Programming (class projects, embedded software engineering internship)
- Grace White, CPRE:
    - Industry experience (internships) in requirements based testing on avionics and developing embedded software on aircraft connection devices.
    - Developing testing suite experience (internship).
- Jessica Reiff, EE:
    - State Space Models (class projects)
    - Embedded Programming (class projects and internship)
- Collin Van Weelden, EE:
    - PCB Schematic and design (EE 2300 and ETG employee)
    - Hardware bench testing (EE 2300)
    - Tons of drone flying and building experience (personal FPV drone)
    - Recreational Drone Pilot License

2. Strategies for encouraging and supporting contributions and ideas from all team members:

The expectations for our team to encourage and support contributions and ideas from all team members are to make sure that all members have opportunities to share their opinions on how we should proceed. Respective team leads should prioritize inclusivity and open-mindedness.

3. Procedures for identifying and resolving collaboration or inclusion issues (e.g., how will

a team member inform the team that the team environment is obstructing their

opportunity or ability to contribute?)

We expect team members experiencing collaboration or inclusion issues to communicate their concerns directly with the team members involved, and to do so with kindness and patience.

Goal-Setting, Planning, and Execution

1. Team goals for this semester:

Have a clear scope and understanding of our project. Have a good start on the project. Work effectively and efficiently in such ways to not grow irritated with each other.

2. Strategies for planning and assigning individual and team work:

Three main teams handle large tasks, and the work is distributed among them based on skills and availability.

3. Strategies for keeping on task:

Keep meetings short in order to avoid off-topic discussions. Have a meeting itinerary to stay on track during meetings.

Consequences for Not Adhering to Team Contract

1. How will you handle infractions of any of the obligations of this team contract?

We expect to handle minor infractions via open and honest communication with one another. Should more extreme infractions occur, instructor intervention may become necessary.

2. What will your team do if the infractions continue?

Report them to our professors and ask for assistance.

****

a) I participated in formulating the standards, roles, and procedures as stated in this contract.

b) I understand that I am obligated to abide by these terms and conditions.

c) I understand that if I do not abide by these terms and conditions, I will suffer the consequences as stated in this contract.

1)	Matthew Smith	DATE: 02/05/2026
2)	Collin Van Weelden	DATE: 02/05/2026
3)	Grace White	DATE: 02/05/2026
4)	Paul Zell	DATE: 02/05/2026
5)	Grace Finger	DATE: 02/03/2026
6)	Jessica Reiff	DATE: 02/03/2026
7)	Chloe Atwood	DATE: 02/03/2026